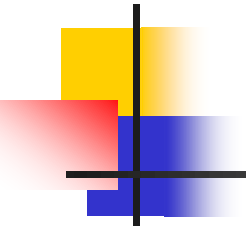
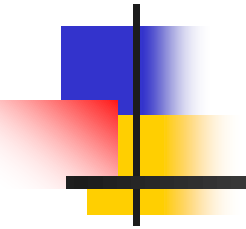
PART 4

DATA STORAGE AND QUERY



Chapter 13

Query Optimization



Main parts in Chapter 13

- § 13.1 Overview
  - why optimization needed
- § 13.2 Transformation of relational expressions
  - equivalence rules
- § 13.4 Choice of evaluation plans — Query optimization
  - cost-based optimization, § 13.4.1/13.4.2
  - ***heuristic optimization,*** § 13.4.3

§ 13.1 Overview

- A SQL query statement may corresponds to several equivalent expressions (*or* query evaluation plans), each of which may have different costs
- ***Query optimization*** is needed to choice an ***efficient*** query-evaluation plan with ***lower*** costs

- E.g.1.

```
select *
from   r, s
where  r.A = s.A
```

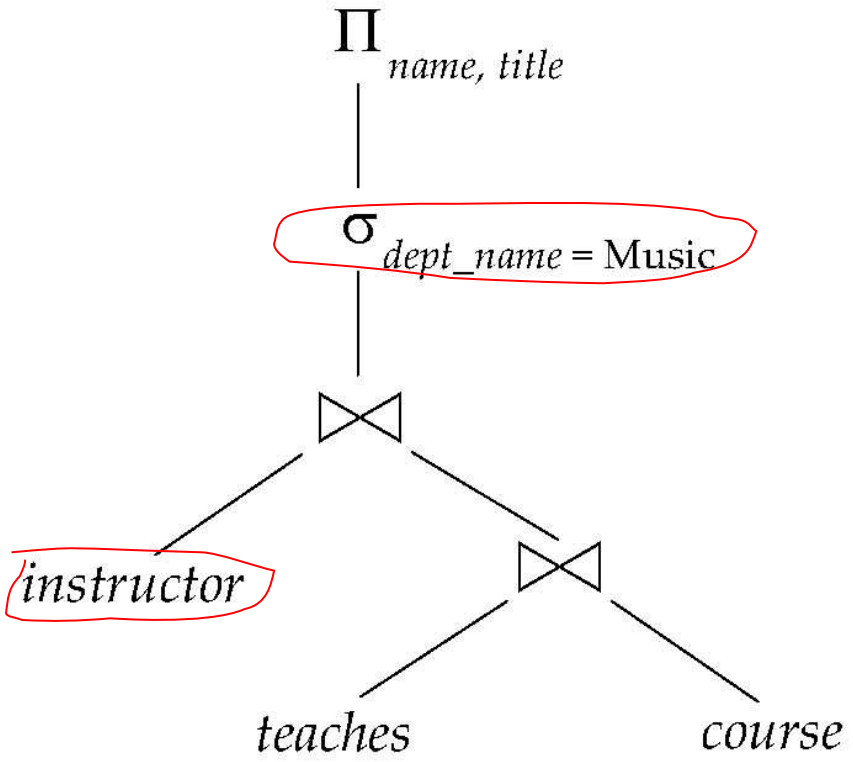
$\boldsymbol{\sigma}_{r.A=s.A}(r \times s)$ is much slower than $r \bowtie s$

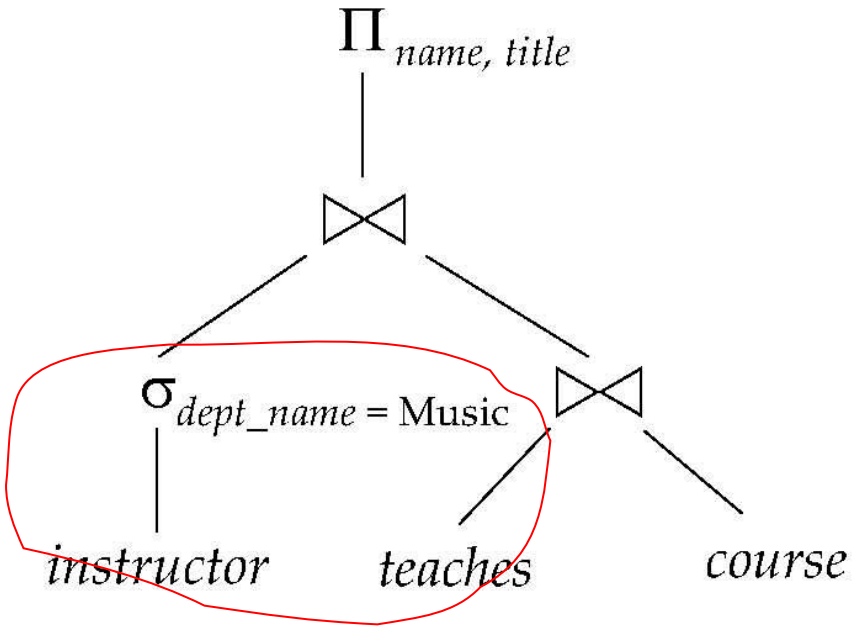
Overview (cont.)

- E.g.2 Find the *names* of all *instructors* in the *Music* department together with the course title of all the courses that the instructors teach

Overview (cont.)

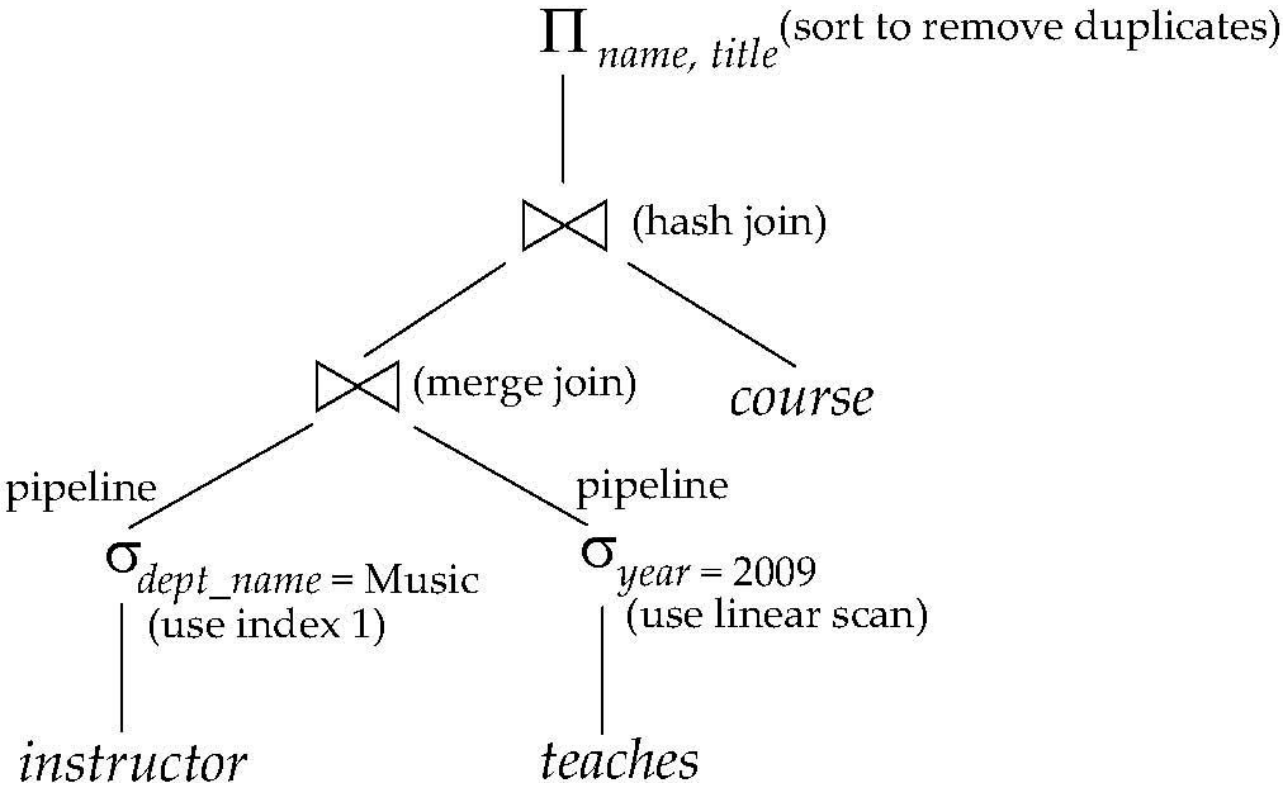
- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation





Overview (cont.)

- An evaluation plan defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.



Overview (cont.)

- Cost difference between evaluation plans for a query can be enormous
  - E.g. seconds vs. days in some cases

Overview (cont.)

- The procedures of optimization
  - generating the equivalent expressions/query trees, by transforming of relational algebra expressions according to equivalence rules in § 13.2
  - generating alternative *evaluation plans*, by annotating the resultant equivalent expressions with implementation algorithms for each operations in the expressions
  - choosing the optimal (that is, cheapest) or near-optimal plan based on the **estimated cost**, by
    - cost-based optimization
    - ***heuristic optimization***

Overview (cont.)

The cost of operations is needed to estimate based on:

- Statistical information about relations. Examples:
  - number of tuples, number of distinct values for an attribute
- statistics estimation for intermediate results
  - to compute cost of complex expressions
- cost formulae for algorithms, computed using statistics

§ 13.2 Transformation of Relational Expressions

- **Definition.** Two relational algebra expressions are equivalent, if on every legal database instances, the two expression generate the same set of tuples

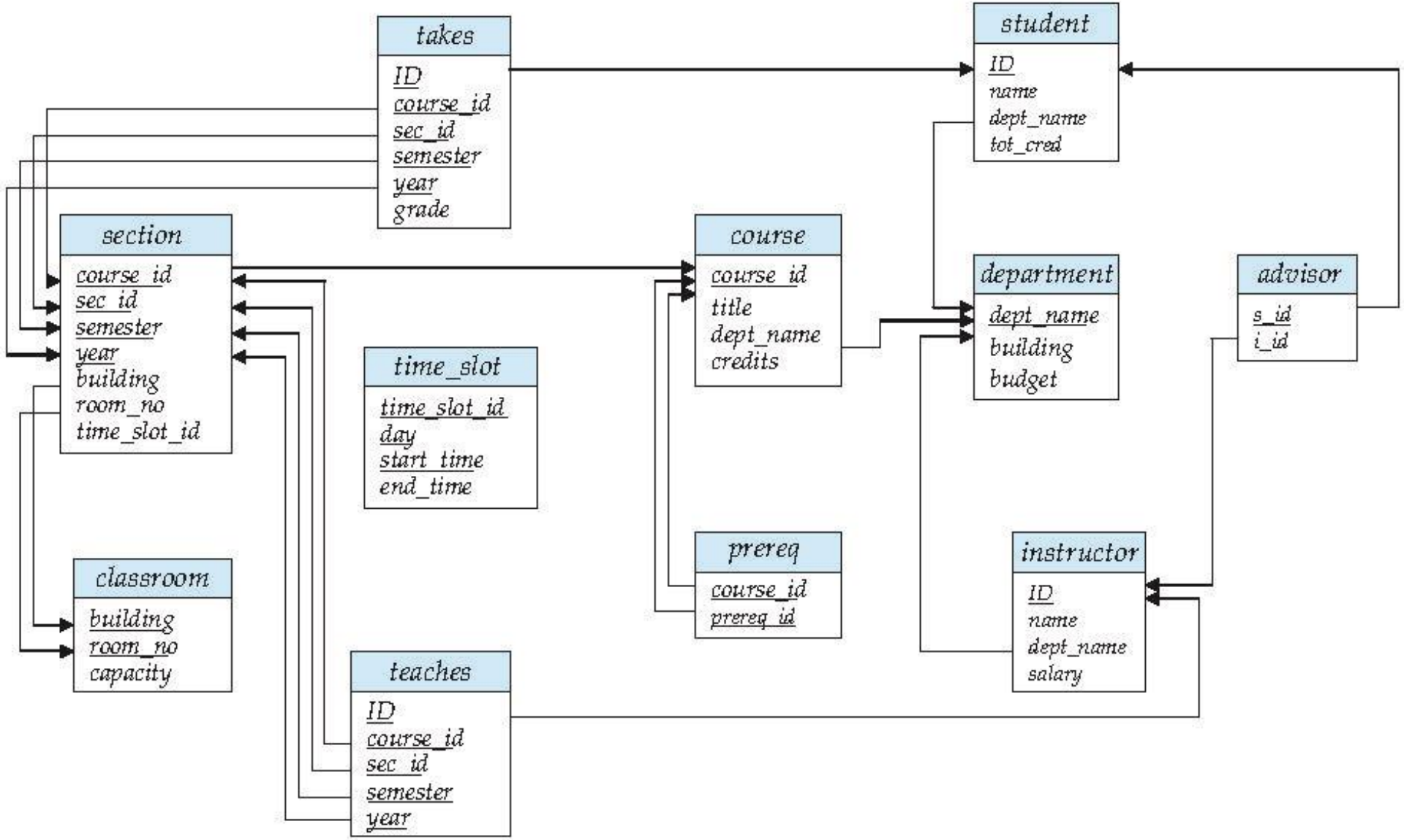


Fig.13.0.1 Schema diagram for the University database

Equivalence Rules (cont.)

- Rule1.(选择串接律, 将1个选择操作分解为2个选择操作) Conjunctive（合取） selection operations can be deconstructed into a sequence of individual selections
- 合取选择运算可分解为单个选择运算的序列。

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

  - e.g. refer to ▷

- Rule2. (选择交换律) Selection operations are *commutative*（交换律）：

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

Equivalence Rules (cont.)

- Rule3. (投影串接律) Only the final operation in a sequence of project operations is needed

$$\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{Ln}(E))\ \ldots)) = \Pi_{L_1}(E)$$

  - note: it is required that $L_1 \subseteq L_2 \subseteq \ldots \subseteq L_n$
  - e.g. $\Pi_{\{ID\}}\{\Pi_{\{ID, name\}}(instructor)\}$
    $= \Pi_{\{ID\}}(instructor)$

Equivalence Rules (cont.)

- **Rule4. (以连接操作代替选择和笛卡尔乘积)** Selections can be combined with Cartesian products and theta joins

  a. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$

  note: definition of **θ** join

  b. $\sigma_{\theta 1}(E_1 \bowtie_{\theta 2} E_2) = E_1 \bowtie_{\theta 1 \wedge \theta 2} E_2$

  - e.g. $\sigma_{\textit{dept-name="Music"}}(\textit{instructor} \bowtie \textit{teaches}) =$

  $\textit{instructor} \bowtie_{(\textit{dept-name="Music"}) \wedge (\text{instructor.ID=teaches.ID})} \textit{teaches}$

- By Rule4, the number of operations can be reduced, and the costs of the right-hand expressions are less than that of the left-hand expressions
  - often used in heuristic query optimizing

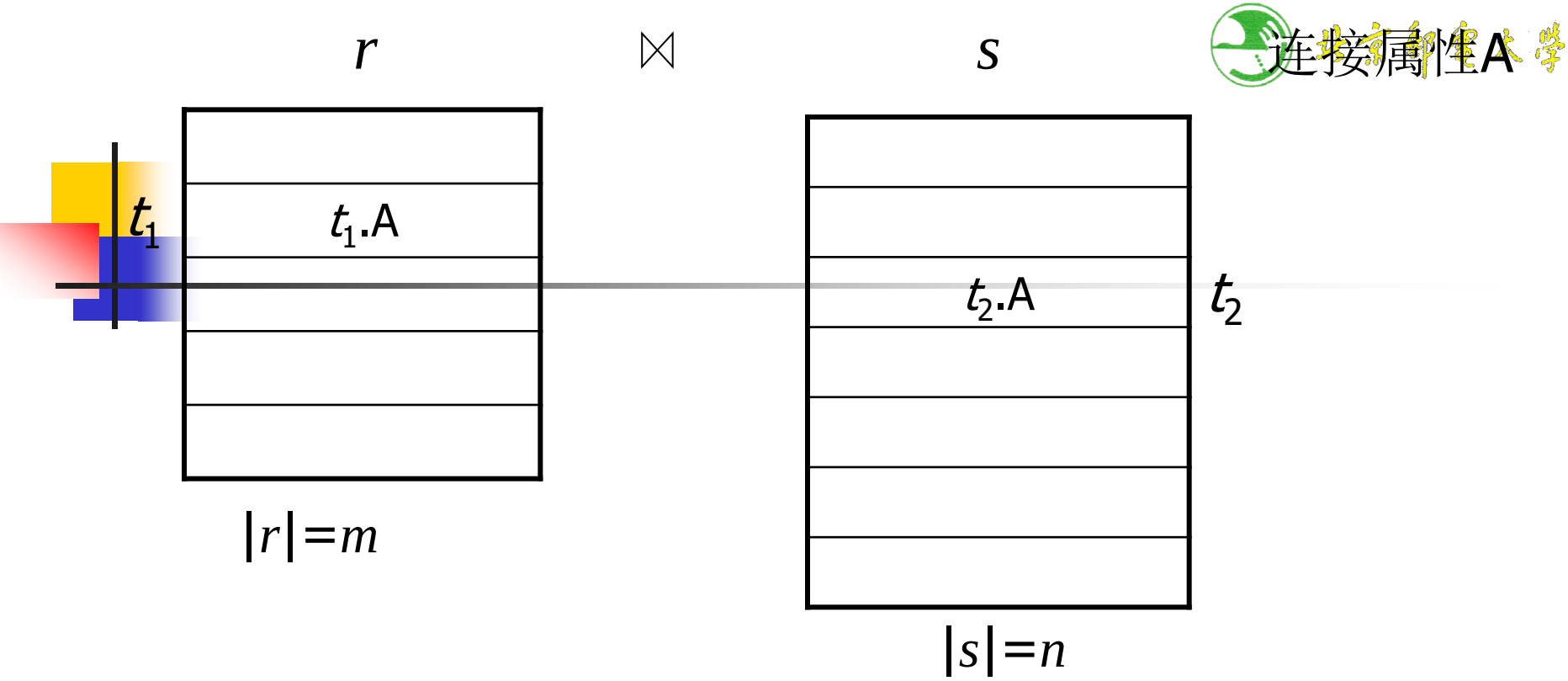
Equivalence Rules (cont.)

- !!! Rule5 (连接操作可交换) Theta join operations are *commutative*

$$E_1 \bowtie_\theta E_2 = E_2 \bowtie_\theta E_1$$

  - Fig. 13.3

- ***!! the expression with smaller size should be arranged as the left one in the operation***



原理： 针对r中元组t_1，检查s中的元组t_2.A，t_1.A=t_2.A？

方法：

For r中每个元组t_1， //*扫描

按照t_1.A，查找s中满足t_1.A=t_2.A的元组t_2，

合并元组t_1和t_2

- 假设
  1. $|r|=m$，$|s|=n$
  2. $m<n$，$n/m=k>1$

- 给定r中的元组t_1，采用B/B$^+$树索引、二分搜索机制，根据t_1.A，查找s中满足t_1.A=t_2.A的元组t_2，所需cost正比于树高，为：

  $O(\lg n)$

  $r \bowtie s$的cost为：　$O(m + m\lg n)$

  $s \bowtie r$的cost为：　$O(n + n\lg m)$

- 如果没有索引：

  $O(m + m * n)$　　vs　　$O(n + n * m)$

$m + m\lg n$ vs $n + n\lg m$

- $m<n$, $n/m = k > 1$, $n = k*m$
- $n\lg m - m\lg n$

  $= k*m\lg m - m(\lg k*m)$

  $= k*m\lg m - m\lg k - m\lg m$

  $= (k-1)m\lg m - m\lg k$

- 一般情况下，$(k-1)m\lg m > m\lg k$
- E.g. $|r| = m = 500$，$|s| = n = 1000$, $k = n/m = 2$

  $(k-1)m\lg m = 500\lg 500 > m\lg k = 500\lg 2$

r: *depositor*, $|r| = 7$

s: *account*, $|s| = 10$

表 - dbo.depositor

customername	accountnumber
Hayes	A-102
Johnson	A-101
Johnson	A-201
Jones	A-217
Lindsay	A-222
Simith	A-215
Turner	A-305
NULL	*NULL*

表 - dbo.account

accountnumber	branchname	balance
A-101	Downtown	500
A-102	Perryridge	400
A-105	Perryridge	500
A-201	Brigh	900
A-215	Mianus	700
A-217	Brigh	750
A-222	Redwood	700
A-305	Round Hill	350
A-306	Downtown	800
A-307	Perryridge	900
NULL	*NULL*	*NULL*

```
select *
from depositor inner join account
    on depositor.accountnumber=account.accountnumber

select *
from account inner join depositor
    on account.accountnumber=depositor.accountnumber
```

YEWEN.DBS实验...Query1.sql* | 表 - dbo.account | 表 - d

```
select *
from depositor inner join account
      on depositor.accountnumber=account.accountnumber

select *
from account inner join depositor
      on depositor.accountnumber=account.accountnumber
```

结果 | 消息

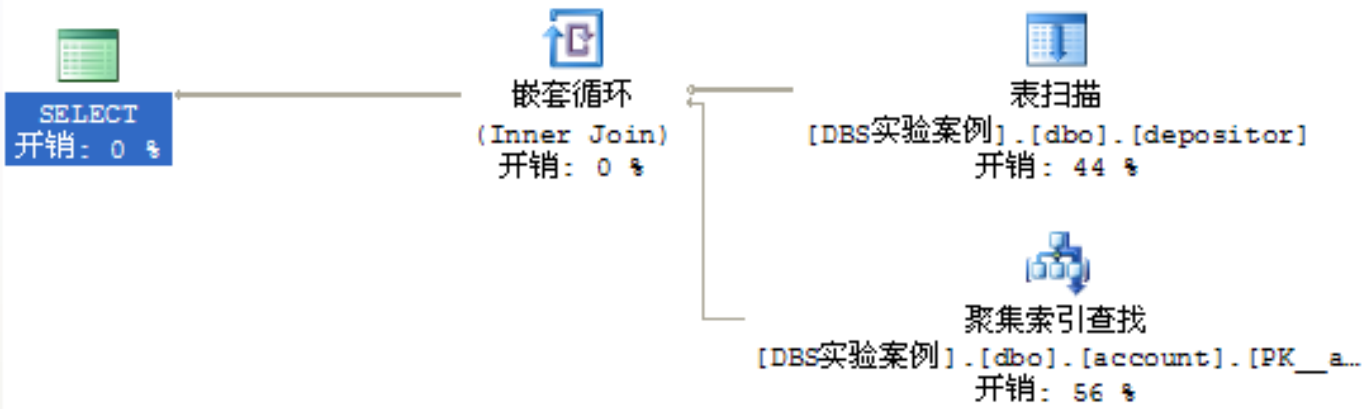
	customemame	accountnumber	accountnumber	branchname	balance
1	Hayes	A-102	A-102	Perryridge	400
2	Johnson	A-101	A-101	Downtown	500
3	Johnson	A-201	A-201	Brigh	900
4	Jones	A-217	A-217	Brigh	750
5	Lindsay	A-222	A-222	Redwood	700
6	Simith	A-215	A-215	Mianus	700
7	Tumer	A-305	A-305	Round Hill	350

	accountnumber	branchname	balance	customemame	accountnumber
1	A-102	Perryridge	400	Hayes	A-102
2	A-101	Downtown	500	Johnson	A-101
3	A-201	Brigh	900	Johnson	A-201
4	A-217	Brigh	750	Jones	A-217
5	A-222	Redwood	700	Lindsay	A-222
6	A-215	Mianus	700	Simith	A-215
7	A-305	Round Hill	350	Tumer	A-305

消息 执行计划

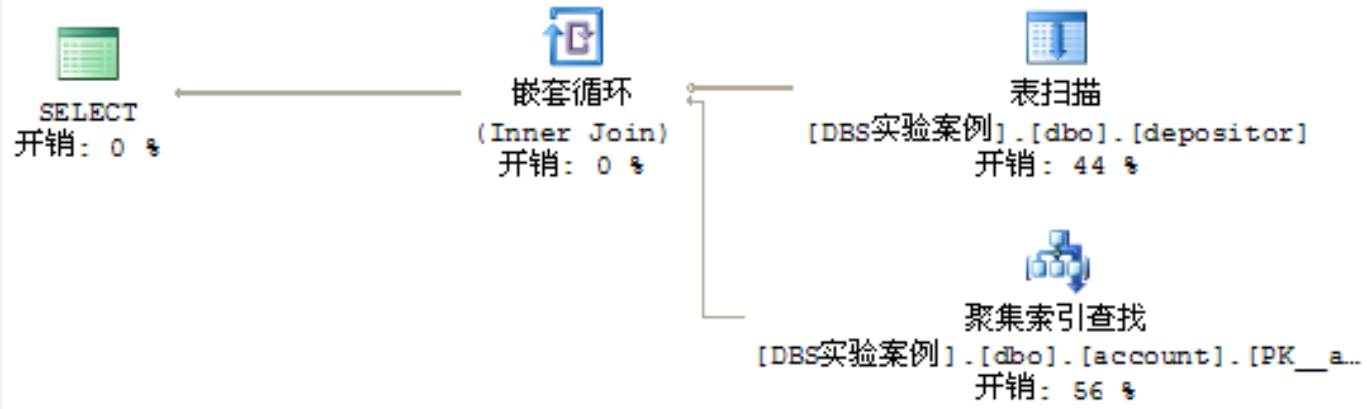
查询 1: (与该批有关的)查询开销: 50%

```
select * from depositor inner join account on depositor.accountnumber=account.accountnumber
```



查询 2: (与该批有关的)查询开销: 50%

```
select * from account inner join depositor on depositor.accountnumber=account.accountnumber
```



SQL Server查询优化器自动选择元组数少的depositor作为连接操作的outer关系，两条语句的查询执行计划、执行成本一样！！！

Innner Join in SQL Server

```
USE AdventureWorks;
GO
SELECT *
FROM HumanResources.Employee AS e
    INNER JOIN Person.Contact AS c
    ON e.ContactID = c.ContactID
ORDER BY c.LastName
```

此内部联接称为同等联接。它返回两个表中的所有列，但只返回在联接列中具有相等值的行。

```
USE AdventureWorks;
GO
SELECT DISTINCT p.ProductID, p.Name, p.ListPrice, sd.UnitPrice AS 'Selling Price'
FROM Sales.SalesOrderDetail AS sd
    JOIN Production.Product AS p
    ON sd.ProductID = p.ProductID AND sd.UnitPrice < p.ListPrice
WHERE p.ProductID = 718;
GO
```

Equivalence Rules (cont.)

- Rule6. (连接操作的结合率, associative)

a. natural join operations are associative

$$(E1 \bowtie E2) \bowtie E3 = E1 \bowtie (E2 \bowtie E3)$$

b. *Theta joins* are associative in the following manner:

$$(E_1 \bowtie_{\theta 1} E_2) \bowtie_{\theta 2 \wedge \theta 3} E_3 = E_1 \bowtie_{\theta 1 \wedge \theta 3} (E_2 \bowtie_{\theta 2} E_3)$$

, where θ_2 involves attributes from only E_2 and E_3

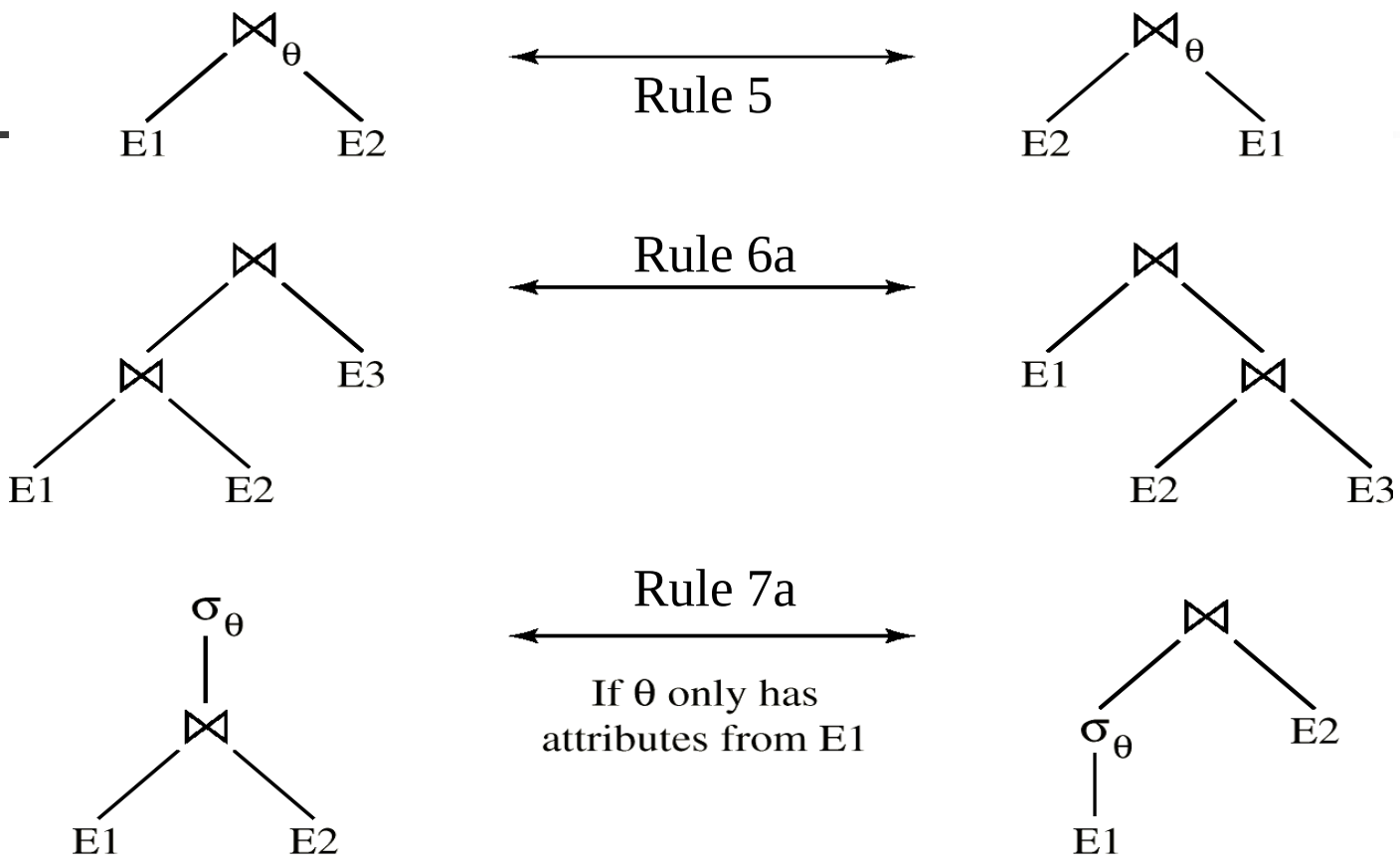


Fig.13.3 Pictorial Depiction of Equivalence Rules

Equivalence Rules (cont.)

- Rule7. Selection operation distributes over theta-join（选择操作对于连接操作的分配率, 选择条件下移）
  - a. when all the attributes in θ_0 involve only the attributes of one of the expressions , e.g. E_1, being joined

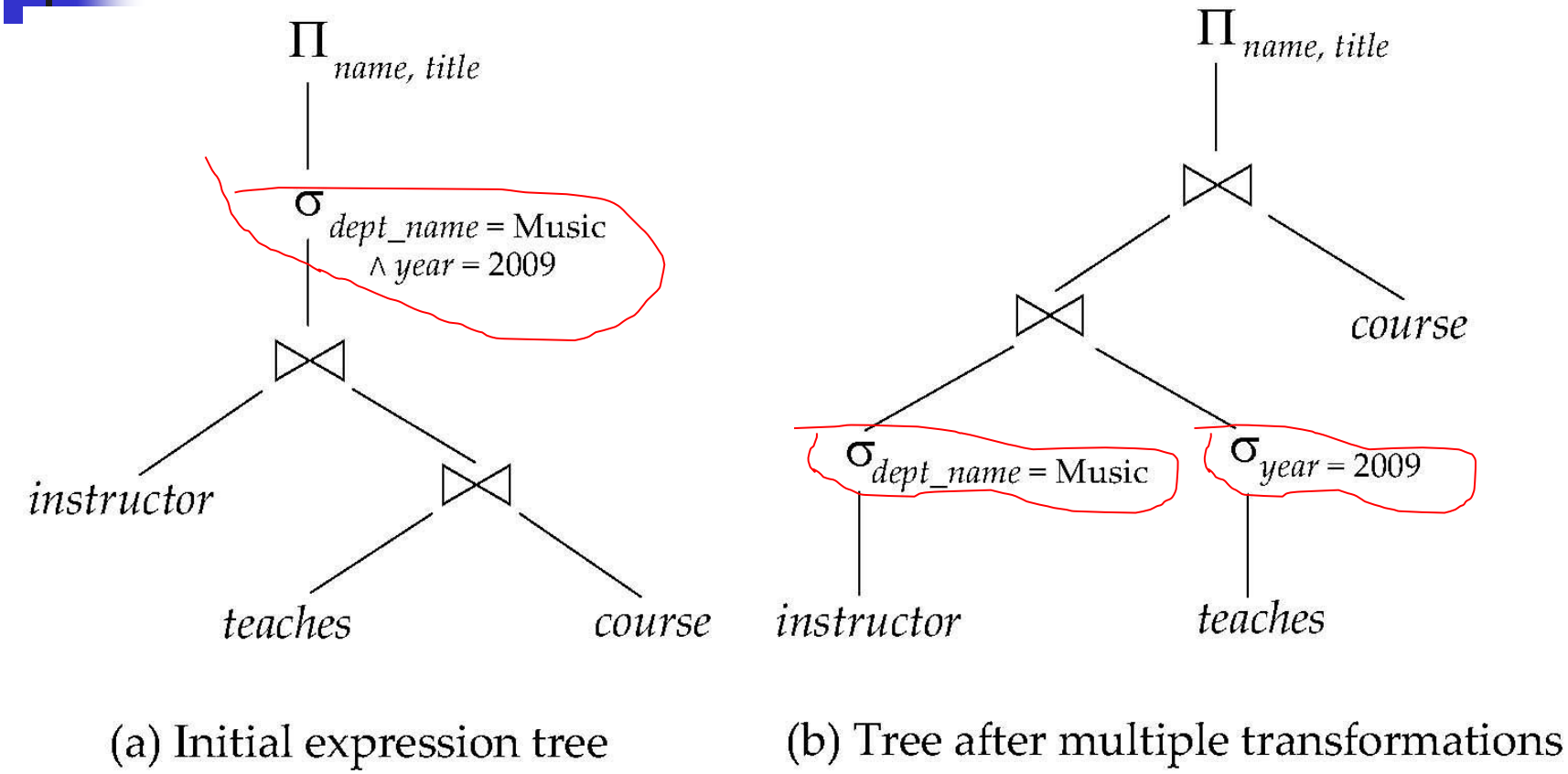
$$\sigma_{\theta 0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 0}(E_1)) \bowtie_{\theta} E_2$$

  - b. when θ_1 involves only the attributes of E_1, and θ_2 involves only the attributes of E_2

$$\sigma_{\theta 1 \wedge \theta 2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 1}(E_1)) \bowtie_{\theta} (\sigma_{\theta 2}(E_2))$$

  - e.g. in Fig.13.0.2

- When Rule7 is applied, the size of relations participating in theta-join operation is reduced, thus evaluation costs are reduced



(a) Initial expression tree (b) Tree after multiple transformations

Fig.14.0.2 An example of Rule7

Equivalence Rules (cont.)

- Rule8. Projection operation distributes over theta-join（投影操作对于连接操作的分配率, 投影下移）
  - a. L_1 and L_2 are the attributes of E_1 and E_2 respectively. if *join condition* θ involves only attributes in $L_1 \cup L_2$, then

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = (\Pi_{L_1}(E_1)) \bowtie_\theta (\Pi_{L_2}(E_2))$$

E1:

instructor (ID, name, dept-name, salary)

E2:

department(dept-name, building, budget)

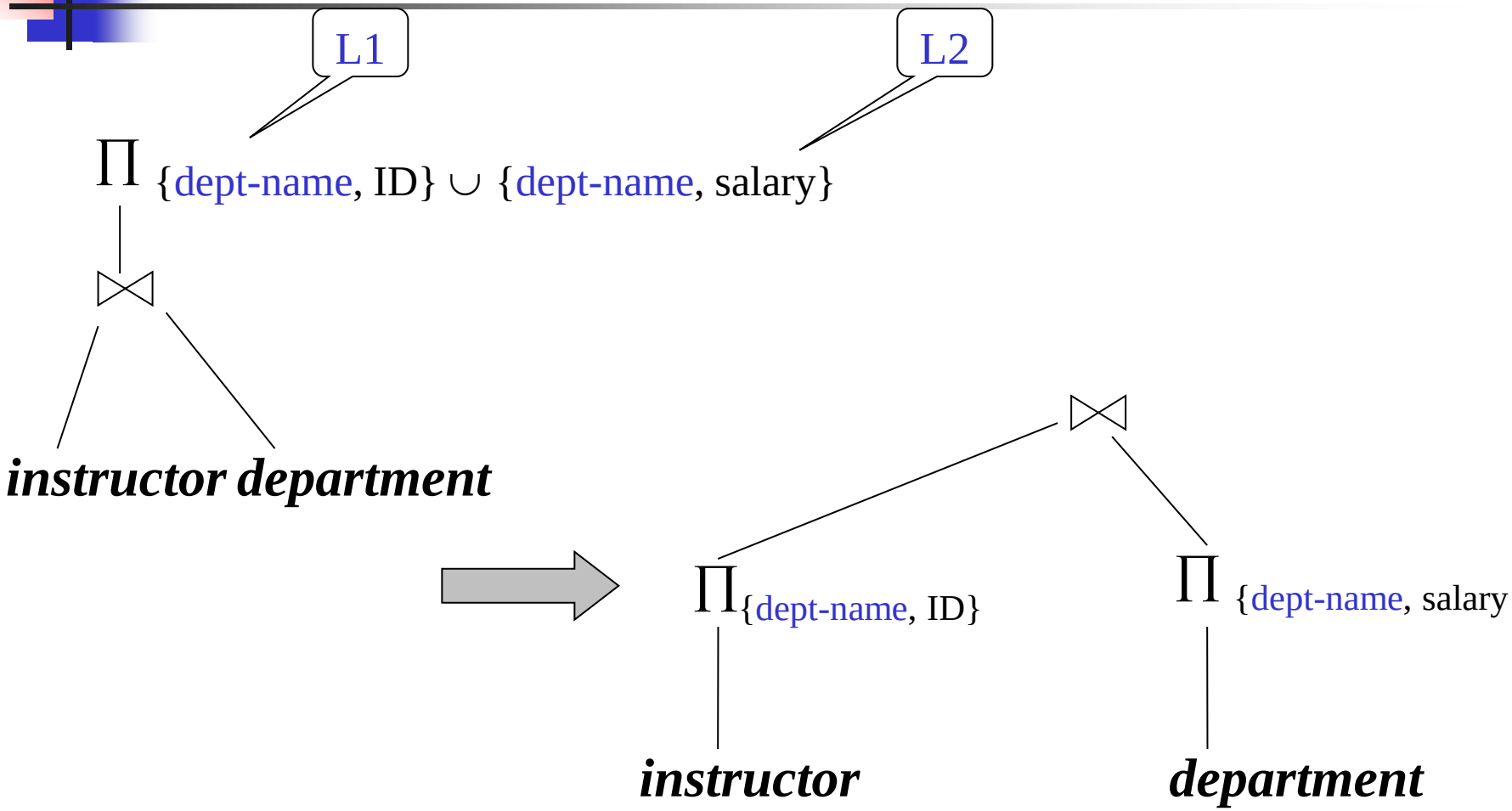


Fig.13.0.3 An example of Rule 8a

Equivalence Rules (cont.)

b. consider a join $E_1 \bowtie_\theta E_2$

- let L_1 and L_2 be sets of attributes from E_1 and E_2, respectively
- let L_3 be attributes of E_1 that are involved in join condition θ, but are not in $L_1 \cup L_2$, and
- let L_4 be attributes of E_2 that are involved in join condition θ, but are not in $L_1 \cup L_2$.

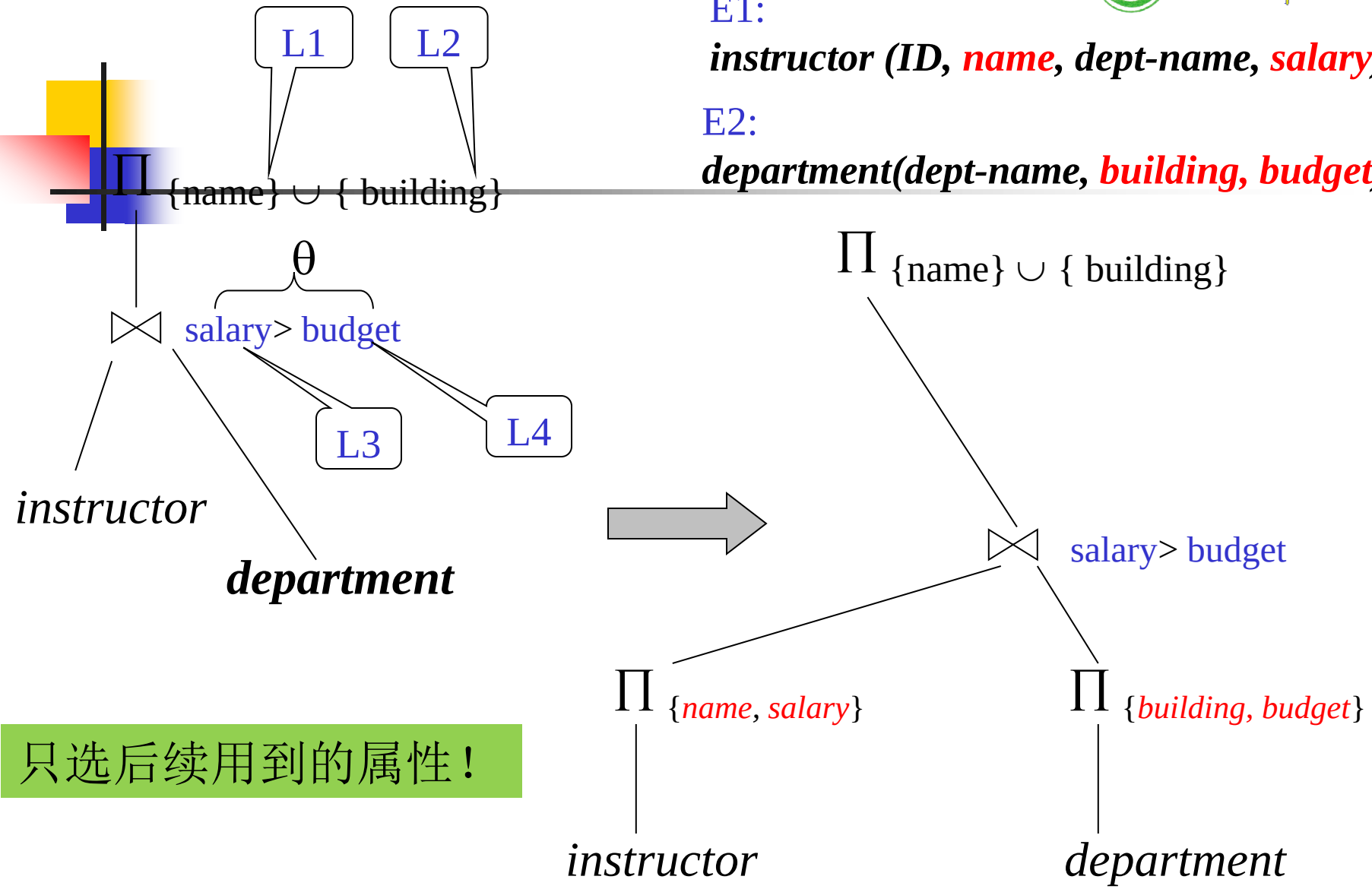
$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_\theta (\Pi_{L_2 \cup L_4}(E_2)))$$

E1:

instructor (ID, name, dept-name, salary)

E2:

department(dept-name, building, budget)



只选后续用到的属性！

Fig.13.0.4 An example of Rule 8b

Equivalence Rules (cont.)

- Rule9. The set operations *union* and *intersection* are commutative　集合并和交满足交换律

$$E_1 \cup E_2 = E_2 \cup E_1$$
$$E_1 \cap E_2 = E_2 \cap E_1$$

set difference is not commutative

- Rule10. Set *union* and *intersection* are associative（结合律）

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$
$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

Equivalence Rules (cont.)

- Rule11. (选择操作对于集合并、交、差的分配率) The selection operation distributes over ∪, ∩ and –.
  - $\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - \sigma_\theta(E_2)$
    - similarly for ∪ and ∩ in place of –
  - $\sigma_\theta (E_1 - E_2) = \sigma_\theta(E_1) - E_2$
    - similarly for ∩ in place of –, but not for ∪

- Rule12. (投影操作对于集合并的分配率) The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

Example One: Pushing Selections

- Query: Find the *names* of all instructors in the *Music* department, along with the titles of the courses that they teach
  - $\Pi_{name,\ title}(\sigma_{dept\_name=\ \text{“Music”}}(instructor \bowtie (teaches \bowtie \Pi_{course\_id,\ title}(course))))$
- Transformation using rule 7a.
  - $\Pi_{name,\ title}((\sigma_{dept\_name=\ \text{“Music”}}(instructor)) \bowtie (teaches \bowtie \Pi_{course\_id,\ title}(course)))$
- Performing the selection as early as possible reduces the size of the relation to be joined.

Example Two: Multiple Transformations

- Find the names of all instructors in the Music department who have taught a course in 2009, along with the titles of the courses that they taught
  - $\Pi_{name,\ title}(\sigma_{dept\_name = \text{“Music”} \wedge year = 2009}(instructor \bowtie (teaches \bowtie \Pi_{course\_id,\ title}(course))))$
- Transformation using join associatively (Rule 6a):
  - $\Pi_{name,\ title}(\sigma_{dept\_name = \text{“Music”} \wedge year = 2009}((instructor \bowtie teaches) \bowtie \Pi_{course\_id,\ title}(course)))$
- Second form provides an opportunity to apply the “perform selections early” rule, resulting in the subexpression

$$\sigma_{dept\_name = \text{“Music”}}(instructor) \bowtie \sigma_{year = 2009}(teaches)$$

Example Three: Pushing Projections

- Consider: $\Pi_{name,\ title}(\sigma_{dept\_name=\ \text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course\_id,\ title}(course))))$
- When we compute

  $(\sigma_{dept\_name\ =\ \text{“Music”}}(instructor) \bowtie teaches)$

  we obtain a relation whose schema is:
  (*ID*, *name*, *dept_name*, *salary*, *course_id*, *sec_id*, *semester*, *year*)
- Push projections using equivalence rules 8a and 8b; eliminate unneeded attributes from intermediate results to get:

  $\Pi_{name,\ title}(\Pi_{\mathbf{name,\ course\_id}}(\sigma_{dept\_name=\ \text{“Music”}}(instructor) \bowtie teaches)) \bowtie \Pi_{\mathbf{course\_id,\ title}}(course))))$
- Performing the projection as early as possible reduces the size of the relation to be joined.

13.2.3 Join Ordering（连接次序）

- For all relations r_1, r_2, and r_3,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

- If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.

Example

- Consider the expression

$$\Pi_{name,\ title}(\sigma_{dept\_name=\ \text{“Music”}}(instructor) \bowtie teaches \bowtie \Pi_{course\_id,\ title}(course))$$

- Could compute $teaches \bowtie \Pi_{course\_id,\ title}(course)$ first, and join result with

$$\sigma_{dept\_name=\ \text{“Music”}}(instructor)$$

but the result of the first join is likely to be a large relation.

- Only a small fraction of the university's instructors are likely to be from the *Music* department
  - it is better to compute

$$\sigma_{dept\_name=\ \text{“Music”}}(instructor) \bowtie teaches$$

first.

略

§ 13.3 Estimating Statistics of Expression Results

- How to estimate the cost of an ***operation p***
  - estimating the size of the resultant relations produced by the operation ***p***, according to statistics in the ***catalog*** about the relations on which the operation is conducted
    - e.g. $\Pi_{\text{customer-name}}(\sigma_{\text{balance}<2500}(account) \bowtie customer)$
  - computing the cost of the operation in terms of *disk access*, by means of the cost formulae shown in chapter13
    - disk access is the predominant cost, and the *number of block transfers from disk* is used as a measure of the actual cost of evaluation.

13.3.1 Statistics in Catalog for Optimization

- n_r: number of tuples in a relation r
- b_r: number of blocks containing tuples of r
- l_r: size of a tuple of r in bytes
- f_r: blocking factor of r
  - the number of tuples of r that fit into one block

- If tuples of r are stored together physically in a file, then:

$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$$

Statistics in Catalog for Optimization (cont.)

- *V(A, r):* number of distinct values that appear in *r* for attribute *A*
  - same as the size of $\prod_A(r)$, $|\prod_A(r)|$

- *Statistics about indices*, such as heights of B^+-trees, that is HT_i, and the number of leaf pages in the indices, are also in catalog

Statistics in Catalog for Optimization (cont.)

- An example for statistical information
  - $n_{account}$ = 10000 (*account* has 10,000 tuples)
  - $f_{account}$ = 20 (20 tuples of *account* fit in one block)
  - *V*(***branch-name, account***) = 50 (50 *branches*)
  - *V*(***balance, account***) = *500* (500 different *balance* values)
  - assume the following indices exist on account
    - a clustered/primary, B^+-tree index for the attribute ***branch-name***
    - a secondary, B^+-tree index for the attribute ***balance***

13.3.2 ***Selection*** Size Estimation

- The size estimation of the result of a ***selection*** operation depends on the *selection predicates*, that is *single equality predicate*, *single comparison predicate*, and *combination of predicates*

- Equality selection $\sigma_{A=a}(r)$
  - assuming uniform distribution of values in relations, and the value ***a*** appears in attribute ***A*** of some record in ***r***
  - the selection is estimated to have
    - $n_r / V(A, r)$ tuples satisfying ***A***=a
    - $\lceil n_r / V(A, r) / f_r \rceil$ blocks that these tuples occupy

Selection Size Estimation (cont.)

- Comparison selections $\sigma_{A \le v}(r)$
  - the lowest and highest values of attributes, that is min(A, r) and max(A, r), are available in catalog
  - the estimated number of records/tuples satisfying comparison condition $A \le V$
    - 0, if $v < \min(A, r)$
    - n_r, if $v \ge \max(A, r)$
    - $n_r . \dfrac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$, otherwise
  - in absence of statistical information, the *cost* is assumed to be $n_r / 2$

Selection Size Estimation (cont.)

- The **selectivity** of a condition θ_i is the ***probability*** that a tuple in the relation *r* satisfies θ_i . If s_i is the number of satisfying tuples in *r*, the selectivity of θ_i is given by s_i / n_r.
- **Conjunction:** $\sigma_{\theta 1 \wedge \theta 2 \wedge \ldots \wedge \theta n}(r)$. The estimate for number of tuples in the result is: $n_r * \frac{s_1 * s_2 * \ldots * s_n}{n_r^n}$
- **Disjunction:** $\sigma_{\theta 1 \vee \theta 2 \vee \ldots \vee \theta n}(r)$. Estimated number of tuples:

$$n_r * \left(1 - (1 - \frac{s_1}{n_r}) * (1 - \frac{s_2}{n_r}) * \ldots * (1 - \frac{s_n}{n_r})\right)$$

- **Negation:** $\sigma_{\neg\theta}(r)$. Estimated number of tuples:

$$n_r - size(\sigma_\theta(r))$$

13.4 Choice of Evaluation Plans — Query optimization

- How to generate the ***evaluation plan*** for a relational algebra expression with lower or the lowest cost
  - generating of the optimized *query trees*
  - selecting
    - implementation algorithms for each operations in the tree
    - *pipeline* or *materialization* strategy for the expression

- E.g. Fig.13.2

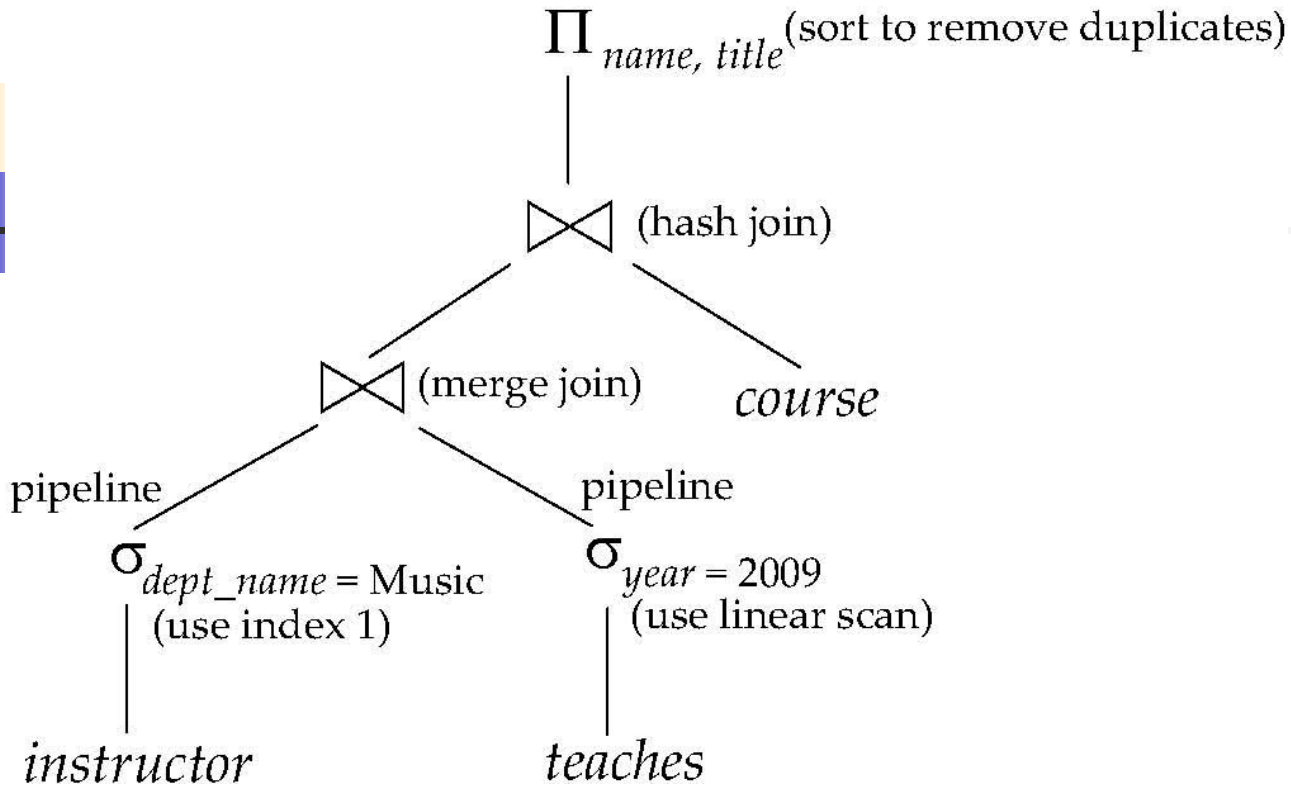


Fig.13.2 An evaluation plan

Choice of Evaluation Plans

- Must consider the interaction of evaluation techniques when choosing evaluation plans
  - choosing the cheapest algorithm for each operation independently may not yield best overall algorithm. E.g.
    - merge-join may be costlier than hash-join, but may provide a sorted output which reduces the cost for an outer level aggregation.
    - nested-loop join may provide opportunity for pipelining

Choice of Evaluation Plans

- **Query optimization**
  - choosing the evaluation plan with ***lowest*** or the ***lower cost***

- Practical query optimizers incorporate elements of the following two broad approaches:

  1. ***Cost-based*** (13.4.1, 13.4.2)

     Search all the plans and choose the best plan in a cost-based fashion.

  2. ***heuristic optimization*** (13.4.3)

     Uses heuristics to choose a plan

- Practical query optimizers incorporate elements of these two approaches

13.4.1 Cost-Based Join Order Selection

- The *join* operator is the basis for multiple table query, and its cost is expensive
- For a relational algebra expression with several *join* operation , the join order influences the query cost of the expression.
- Consider finding the best join-order for $r_1 \bowtie r_2 \bowtie \ldots r_n$.
- There are $(2(n-1))!/(n-1)!$ different join orders for above expression. With $n = 7$, the number is 665280, with $n = 10$, the number is greater than 176 billion!
- No need to generate all the join orders. Using dynamic programming, the least-cost join order for any subset of $\{r_1, r_2, \ldots r_n\}$ is computed only once and stored for future use.

Join Order Selection by Dynamic Programming

- No need to generate all the join orders. Using dynamic programming, the least-cost join order for any subset of $\{r_1, r_2, \ldots r_n\}$ is computed only once and stored for future use

- 方法：

  P600, Fig.13.7;

- 类似于《算法设计与分析》中的矩阵连乘积

$(A((BC)D))$ $(A(B(CD)))$ $((AB)(CD))$

$(((AB)C)D)$ $((A(BC))D)$

13.4.2 Cost-based Optimization with Equivalence Rules

- Cost-based optimization
  - search ***all??*** the possible plans and choose *the best* plan
  - a ***optimal*** plan with the minimum costs can be obtained
- Cost-based optimization is expensive

- **Physical equivalence rules** allow logical query plan to be converted to physical query plan specifying what algorithms are used for each operation.

Cost-based Optimization with Equivalence Rules

- Efficient optimizer based on equivalent rules depends on
  - A space efficient representation of expressions which avoids making multiple copies of subexpressions
  - Efficient techniques for detecting duplicate derivations of expressions
  - A form of dynamic programming based on **memoization**, which stores the best plan for a subexpression the first time it is optimized, and reuses in on repeated optimization calls on same subexpression
  - Cost-based pruning techniques that avoid generating all plans
- Pioneered by the Volcano project and implemented in the **SQL Server** optimizer

13.4.3 Heuristic Optimization

- Heuristic optimization
  - uses ***heuristics*** (or ***heuristic rules***) to choose *a better* plan
  - to reduce the number of choices that must be made in a cost-based fashion
  - a ***suboptimal*** plan with lower costs can be obtained

- Principles

  transforms the query-tree by heuristics to reduce cost
  - perform ***selection*** early to reduce the number of tuples
  - perform ***projection*** early to reduces the number of attributes
  - substitute ***Cartesian product*** and ***selection*** with ***join***

Heuristic Optimization (cont.)

- perform ***most restrictive selection*** and ***join*** operations before other similar operations
  - the *most restrictive* operations generate resulting relations with smallest size

- Heuristic optimization used in some versions of **Oracle**

Steps in Heuristic Optimization

- Step1 使用rule1(▶)，将conjunctive selection 分解为多个单独的*选择操作*，以使单个选择操作尽可能沿查询树下移（尽早执行选择操作，以减少中间计算结果）
- Step2 根据*选择操作*的交换率和分配率，利用rule2, rule7.a, rule7.b, rule11， 将查询树上的每个选择操作尽可能移向叶节点，以便尽早执行选择操作
- ***Step3*** 根据*连接操作*的结合律和交换率，使用rule6(▶)，重新安排查询树中的叶结点，使得具有***restrictive selection***特征的叶结点先执行
  - *restrictive selection*：执行此操作后，产生的结果关系最小（所含元组最少）

Steps in Heuristic Optimization (cont.)

- Step4 利用rule4.a(▶), 以*连接操作*代替相邻的选择和笛卡尔乘积操作

- Step5 利用rule3, 8.a, 8.b,12, 将查询树上的*投影操作*尽可能下移，以便尽早执行投影操作，减少中间计算结果

- Step6 将最后的查询树分解为多个子树，使子树中的各操作可以采用*流水线方式*执行，以减少对外设的访问次数
  - *e*.g. Fig.13.2 ▷

重点：前5步！！！

Example One

- Given
  - S(S#, sname, age, sex)
  - C(C#, cname, teacher)
  - SC(S#, C#, grade)
- Find all the ***students***, who are ***male***, and get ***grades*** more than 90 when they learn some one ***course***
- Step1. SQL statement
  - SELECT DISTINCT *sname*
    FROM *S*, *SC*
    WHERE *S.s# = SC.s#* AND *sex=M* AND *grade > 90*

selection conditions

join condition

Example One (cont.)

- Principles

  select $A1, A2, \ldots, An$
  from $r_1, r_2, \ldots, r_n$
  where P

  corresponds to

  $$\prod_{A1, A2, \ldots, An} (\sigma_P (r_1 \times r_2 \times \ldots \times r_n))$$

Example One (cont.)

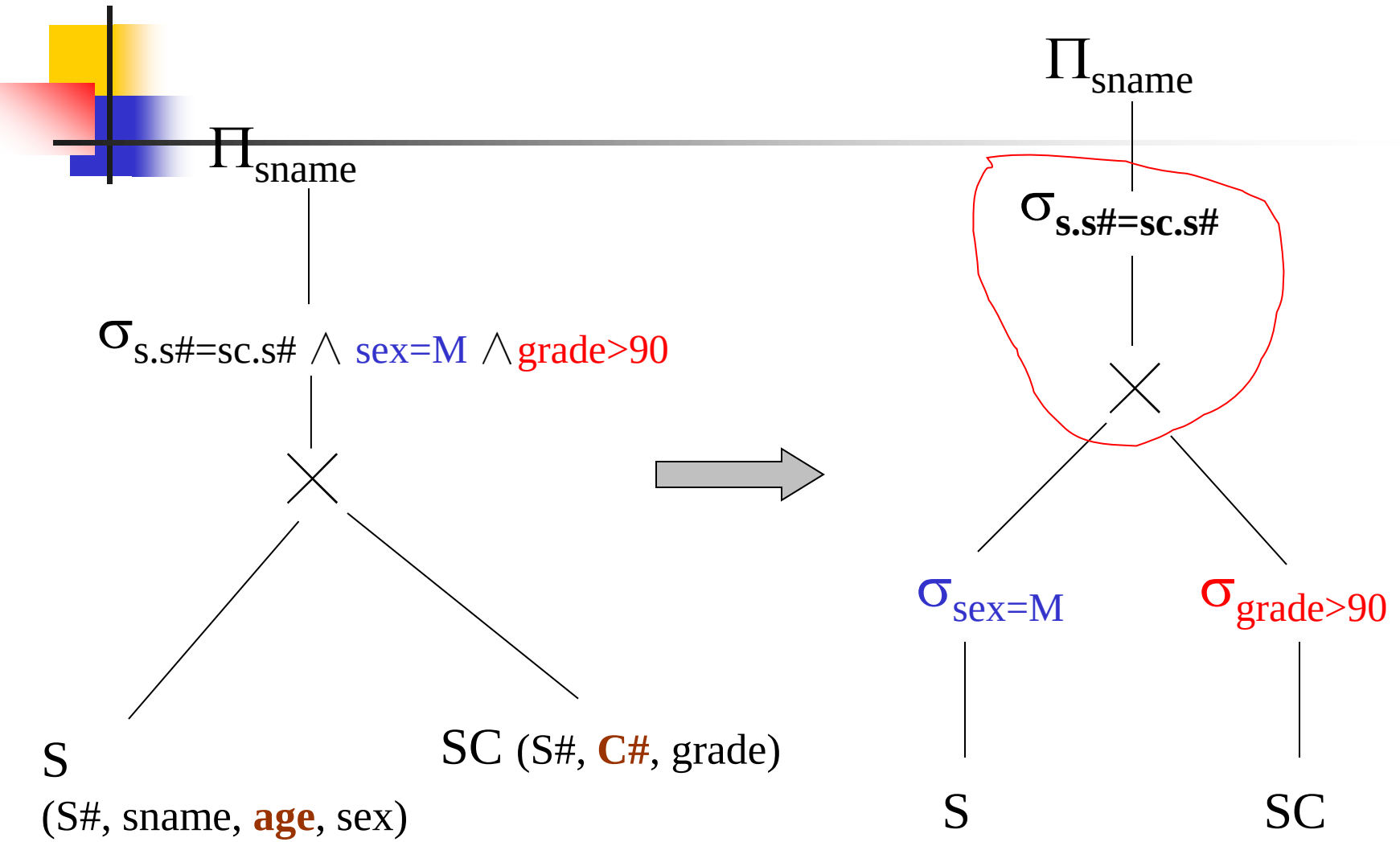
- Step2. Initial Relational algebra expression
  - E1 =

    $\Pi_{sname}(\sigma_{s.s\#=sc.s\# \wedge sex=M \wedge grade>90}(S \times SC))$

或：$\Pi_{sname}(\sigma_{sex=M \wedge grade>90}(S \bowtie SC))$

- Step3. Initial query tree
  - Fig. 14.0.5 (a), E1

- Step4. By means of *Rule1, Rule7b*, distribute σ operations over relation ***S*** and ***SC***
  - Fig. 13.0.5 (b), E2



(a) Initial query tree

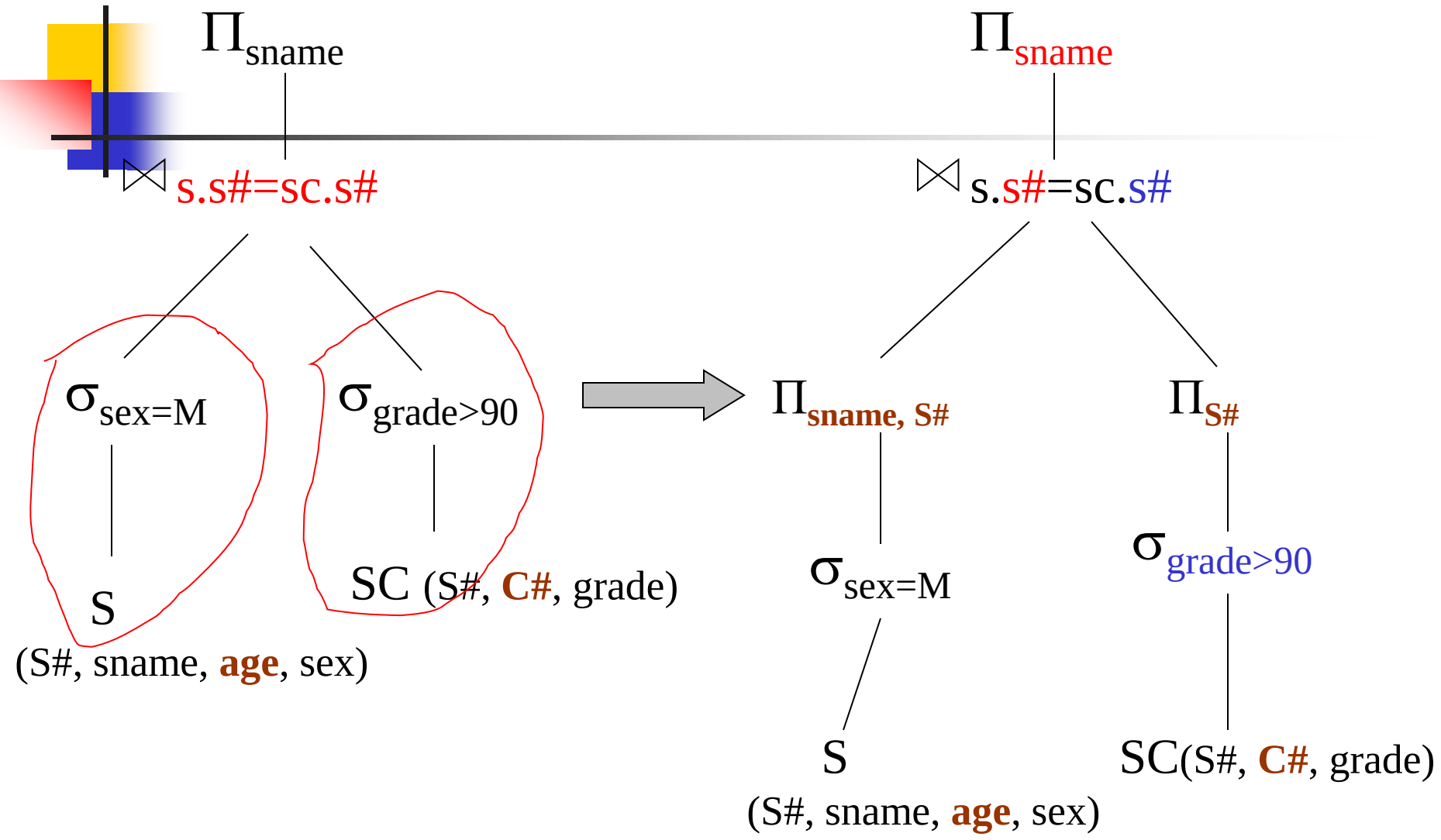
E1

（b）E2

Example One (cont.)

- Step5. By means of *Rule4.a*, replace *selection* and *Cartesian product* with *natural join*
  - Fig. 13.0.5 (c), E3

- Step6. !! By means of *Rule8b*, distribute Π operations over relation ***S*** and ***SC***
  - Fig. 13.0.5 (d),
- The final optimized expression E4 that is equivalent to initial expression E1 is
  - $\Pi_{sname}\{\Pi_{sname,\ S\#}(\sigma_{sex=M}(S)) \bowtie \sigma_{grade>90}(\Pi grade,\ S\#(SC))\}$


Π sname
⋈ s.s#=sc.s#
σ sex=M
σ grade>90
S
(S#, sname, age, sex)
SC (S#, C#, grade)
Π sname, S#
σ sex=M
S
(S#, sname, age, sex)
Π S#
σ grade>90
SC(S#, C#, grade)

（c）E3

（d）E4

Example Two

- Consider the following insurance database, where the primary keys are underlined,
  - ***Person***(driver-id, name, address)
  - ***Car***(license, model, year)
  - ***Accident***(report-number, date, location)
  - ***Participated***(driver-id , license, report-number, damage-amount)

Example Two (cont.)

- Problems
  - Give a SQL statement to find out the ***driver name***, ***license***, ***report-number*** of the ***accidents*** which happened in *Beijing* and *before May 3, 2008*
  - Give the initial query tree for this query, and construct an optimized and equivalent *relational algebra expression* for it by means of heuristic optimization

Example Two (cont.)

- Step1. SQL statement
  - **Select** *name, license, Accident.report-number*

    From *Person, Participate, Accident*

    Where Person.driver_id = Participate.driver_id **AND**

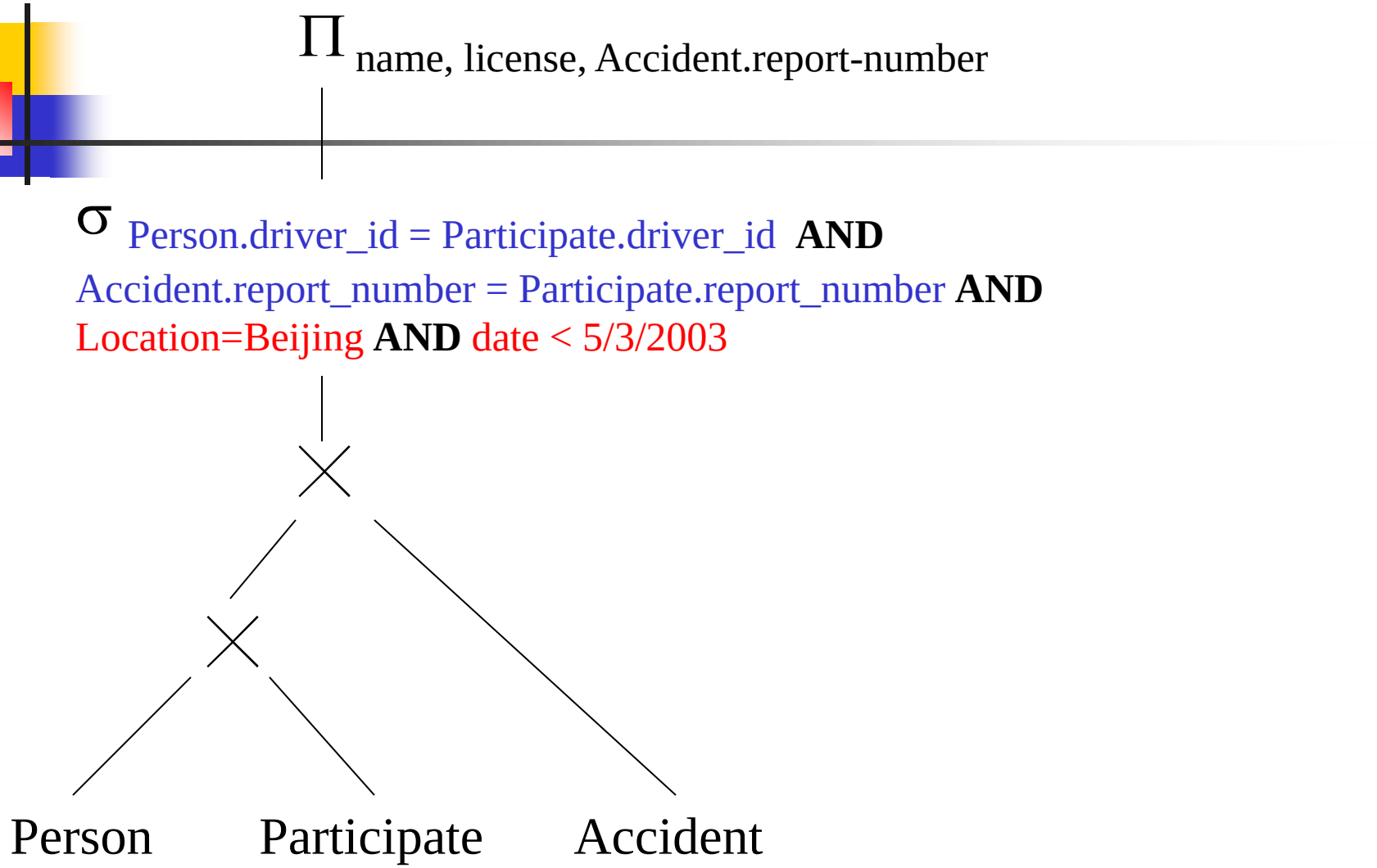
    Accident.report_number = Participate.report_number

    AND Location=Beijing AND date < 5/3/2008
- Step2. Initial expression

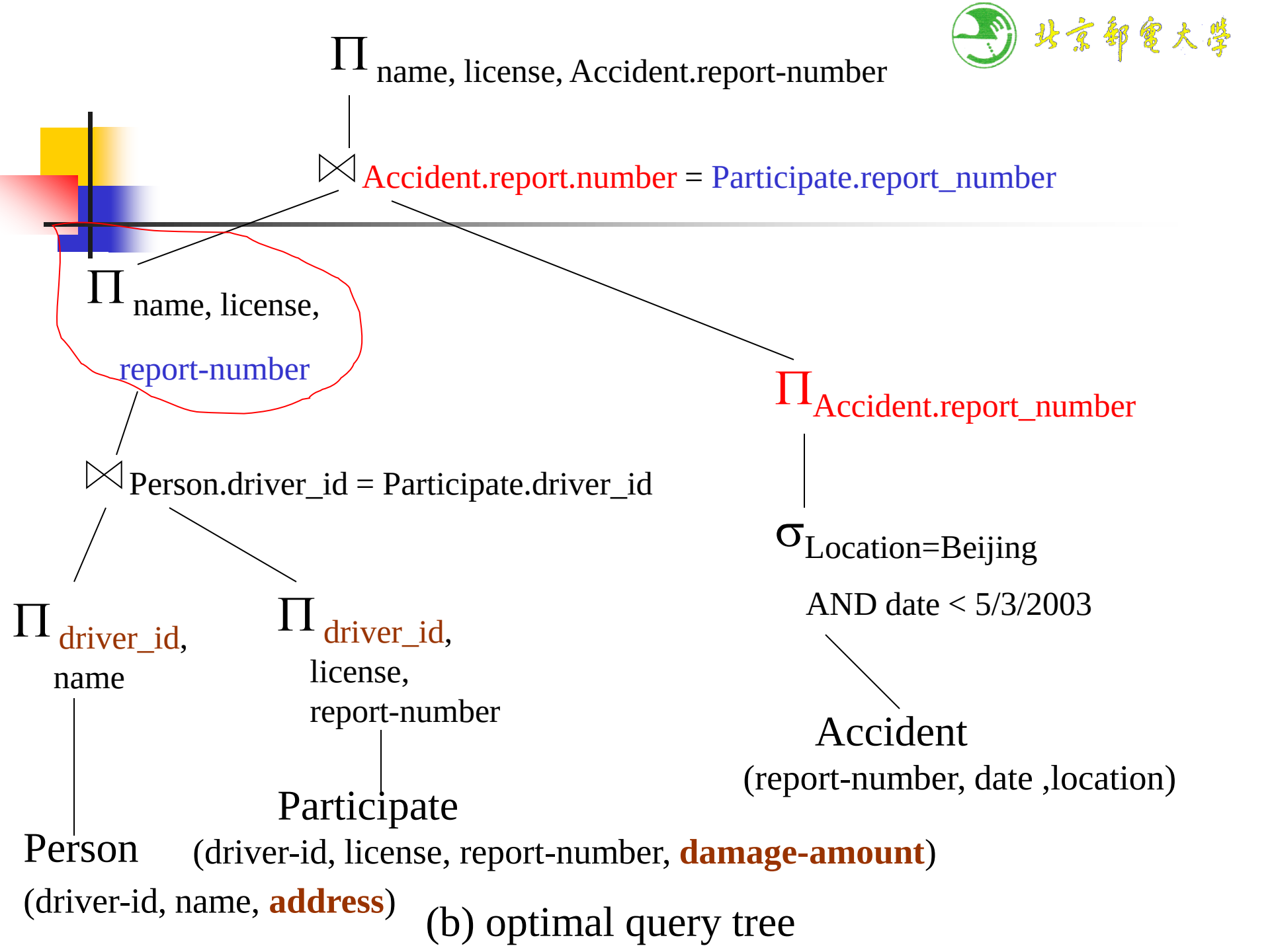
$\Pi_{\text{name, license, Accident.report-number}}$

$(\sigma_{\text{Person.driver\_id = Participate.driver\_id AND Accident.report\_number = Participate.report\_number AND Location=Beijing AND date < 5/3/2003}}$

$(\text{Person} \times \text{Participate} \times \text{Accident})$



(a) initial query tree



(b) optimal query tree

Example Three

- Consider the following relations in banking enterprise database, where the primary keys are underlined
  - *branch (branch-name, branch-city, assets),*
  - *loan (loan-number, branch-name , amount)*
  - *borrower(customer-name, loan-number, borrow-date)*
  - *customer (customer-name, customer-street, customer-city)*
  - *account (account-number, branch-name, balance)*
  - *depositor (customer-name, account-number , deposit-date)*

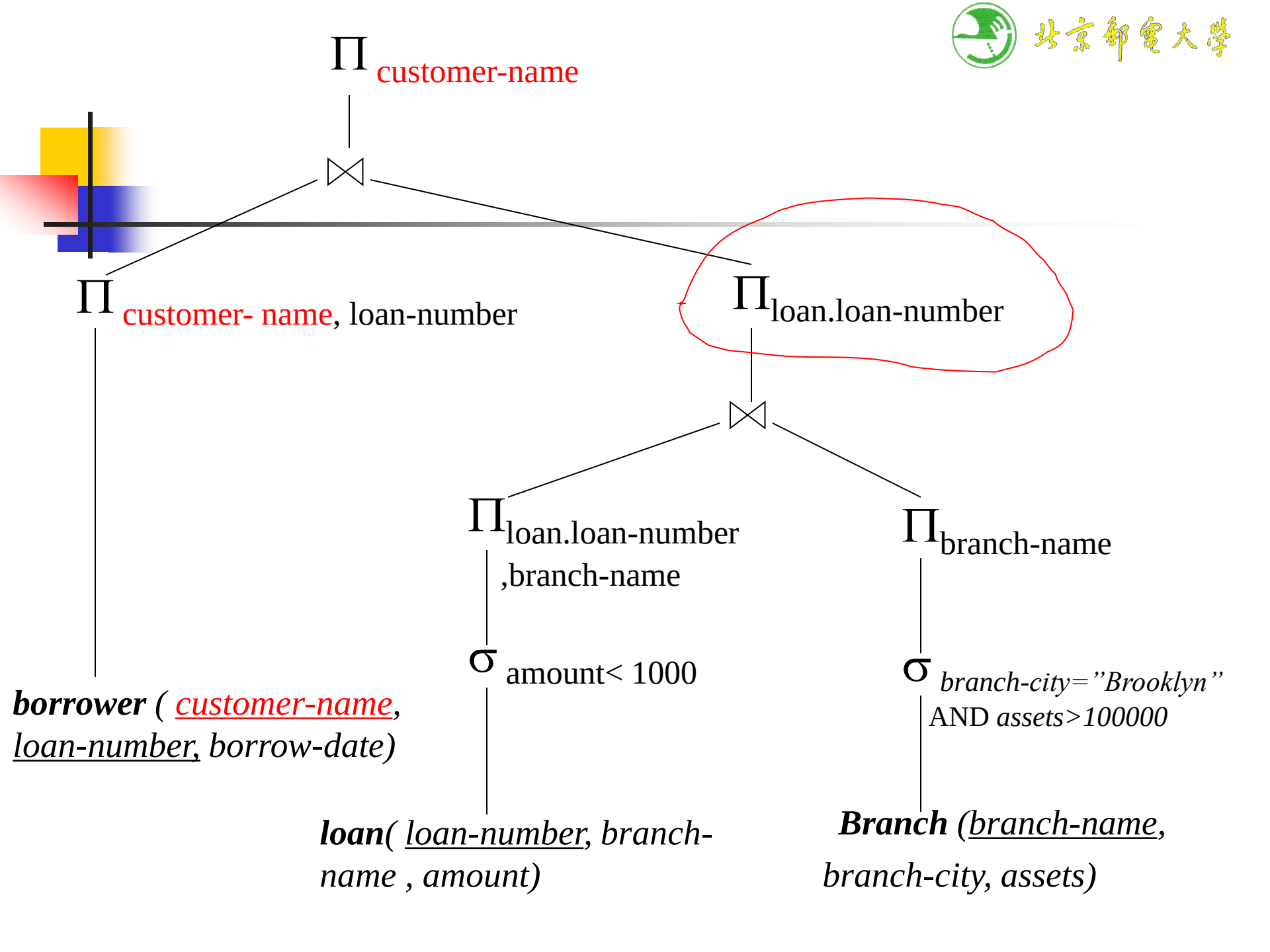
Example Three (cont.)

- For the query " Find the ***names*** of all ***customers*** who have an ***loan*** at any ***branch*** that is located in ***Brooklyn*** and have ***assets*** more than $100,000, requiring that ***loan-amount*** is less than $1000"
  - give an SQL statement for this query
  - given a initial query tree for the query, and convert it into an optimized query tree by means of heuristic optimization

Example Three (cont.)

- SQL

  select *customer-name*
  from *borrower*, *loan*, *branch*
  where *loan.loan-number=borrower.loan-number*
  and *branch.branch-name=loan.branch-name*
  and *branch-city=”Brooklyn”*
  and *assets>100000 and amount<1000*

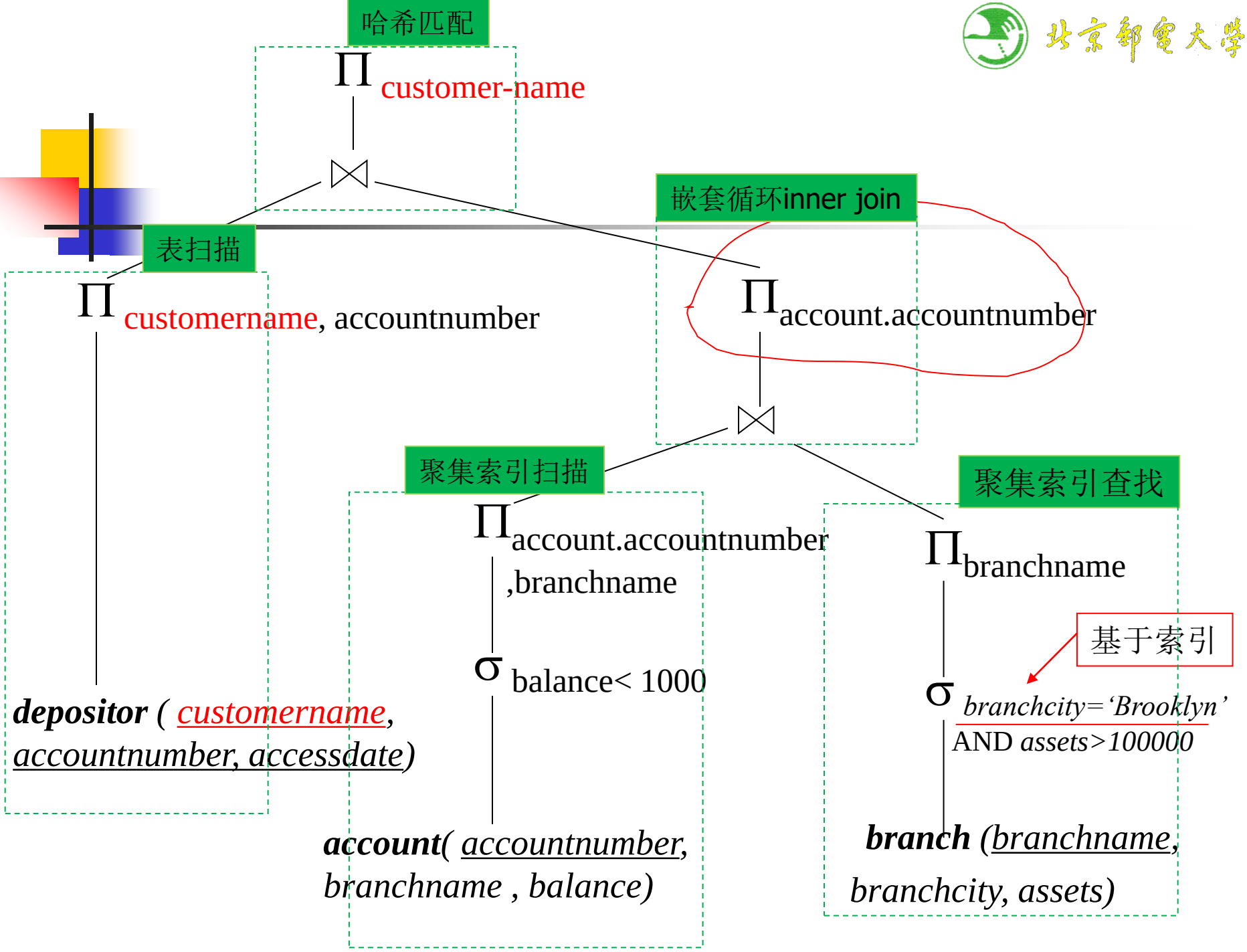

Π customer-name
Π customer- name, loan-number
Π loan.loan-number
Π loan.loan-number ,branch-name
Π branch-name
σ amount< 1000
σ branch-city="Brooklyn" AND assets>100000
borrower (customer-name, loan-number, borrow-date)
loan(loan-number, branch-name , amount)
Branch (branch-name, branch-city, assets)

实际DBS中的查询优化

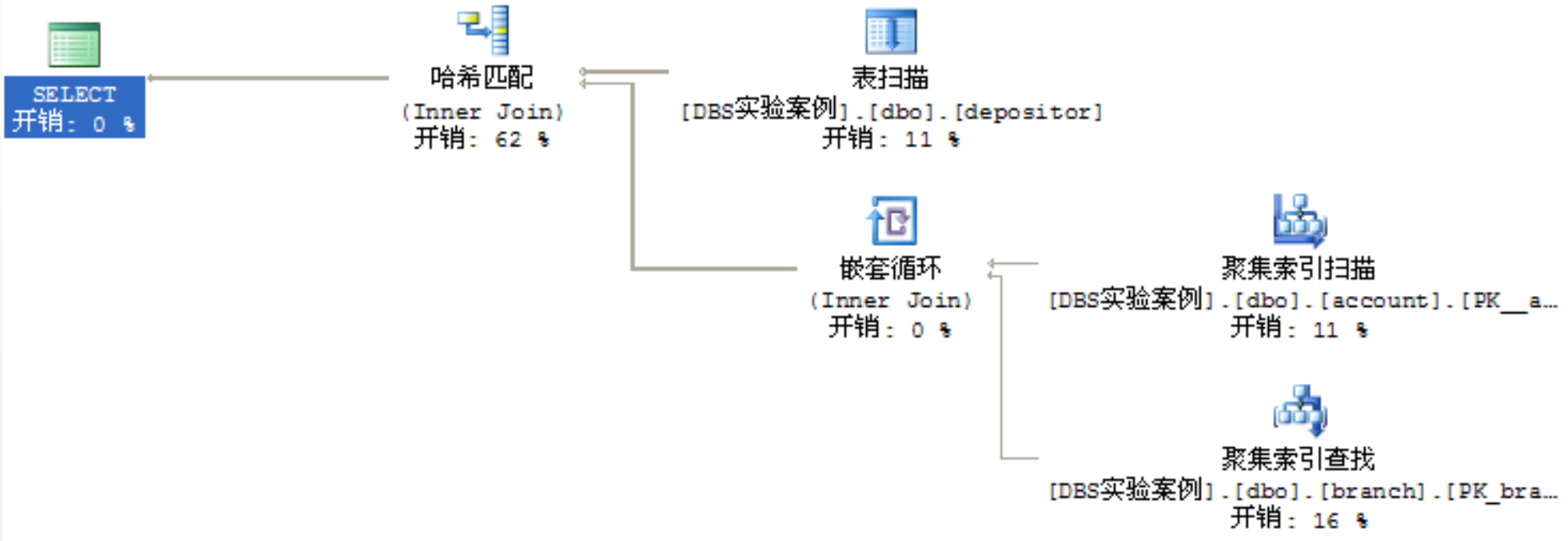
- 教科书上的选择、投影操作下移的实现方式

  实际平台下（如SQL Server）与后面的连接操作结合在一起做，避免多次关系代数操作：

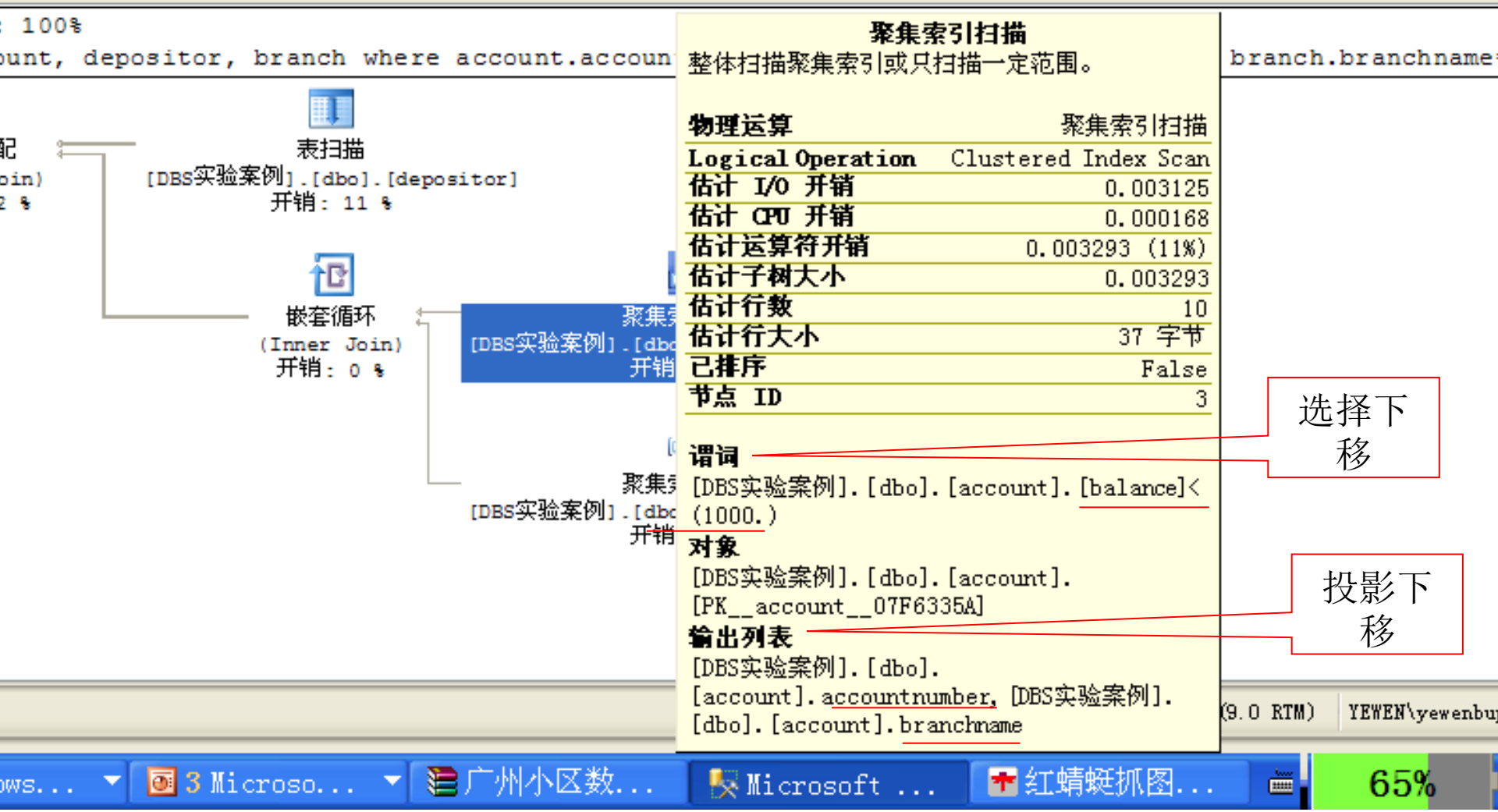
  1. 采用pipeline，在对参与连接运算的关系扫描过程中完成符合条件的元组—选择
  2. 再选出后续步骤需要的属性—投影

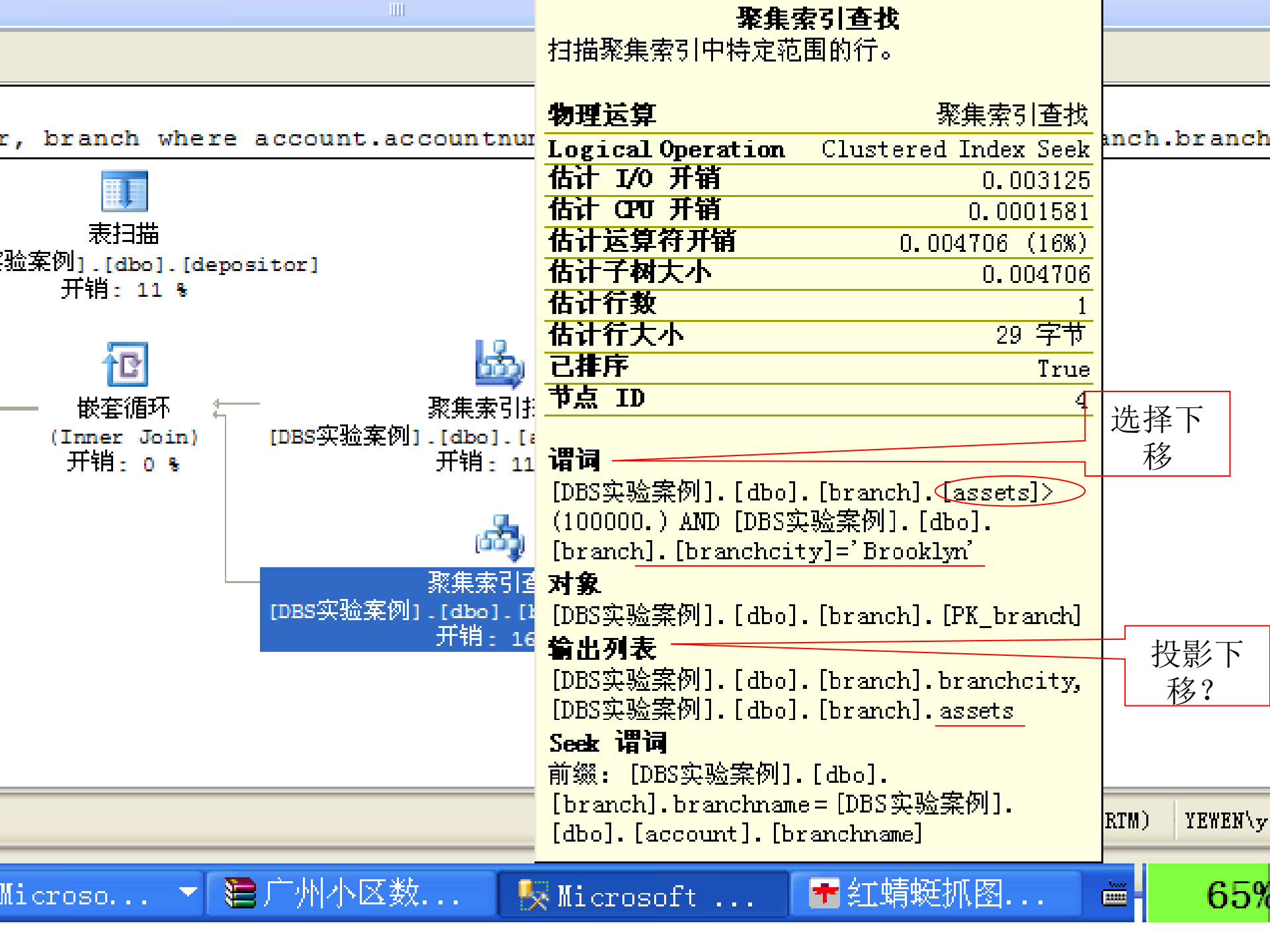

哈希匹配
Π customer-name
⋈
表扫描
嵌套循环inner join
Π customername, accountnumber
Π account.accountnumber
depositor (customername, accountnumber, accessdate)
⋈
聚集索引扫描
聚集索引查找
Π account.accountnumber ,branchname
σ balance< 1000
account(accountnumber, branchname , balance)
Π branchname
基于索引
σ branchcity='Brooklyn' AND assets>100000
branch (branchname, branchcity, assets)

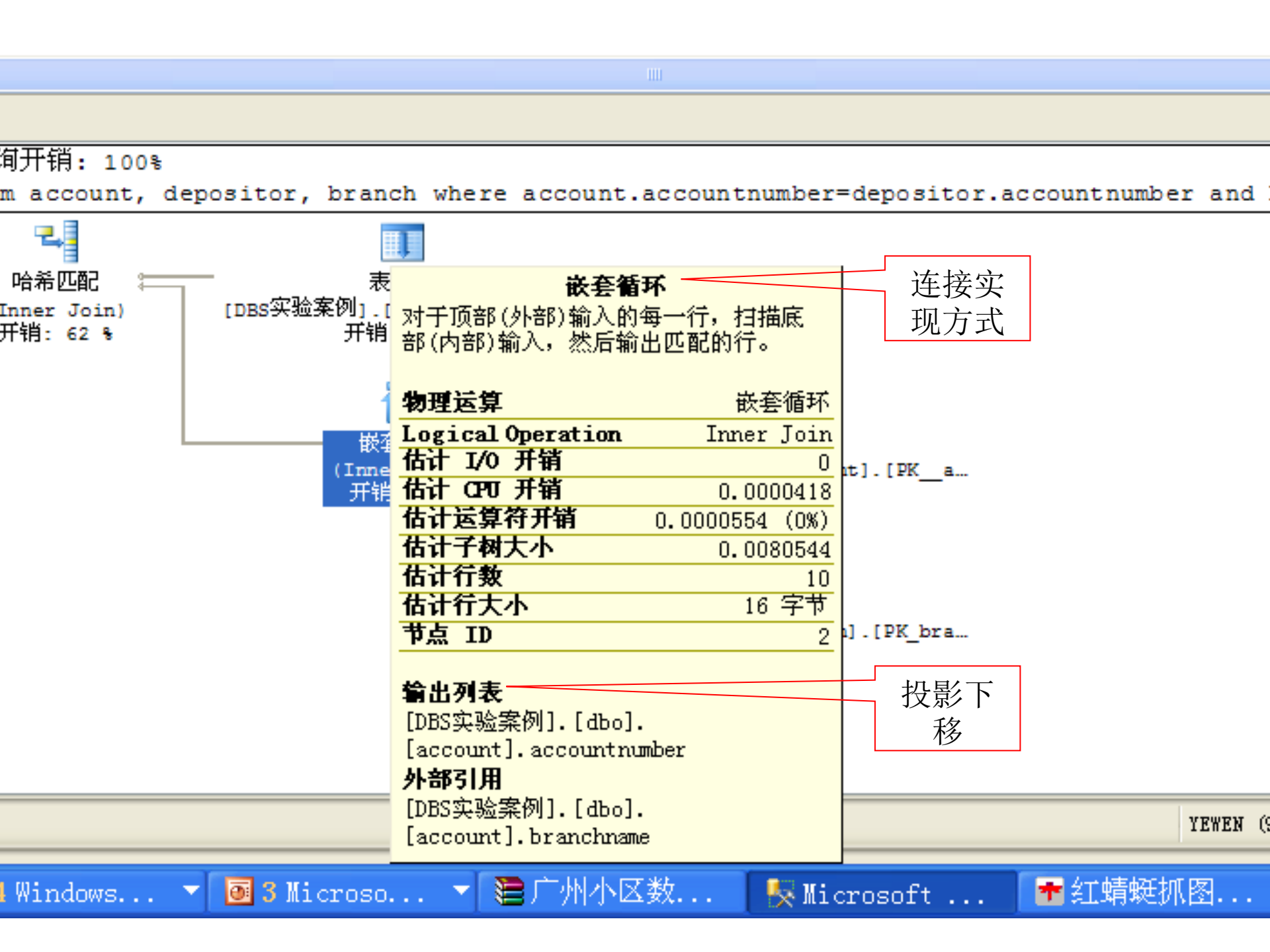
```
select customername
from  account, depositor, branch
where account.accountnumber=depositor.accountnumber
    and branch.branchname=account.branchname
    and branchcity='Brooklyn'
    and assets>100000 and balance<1000
```

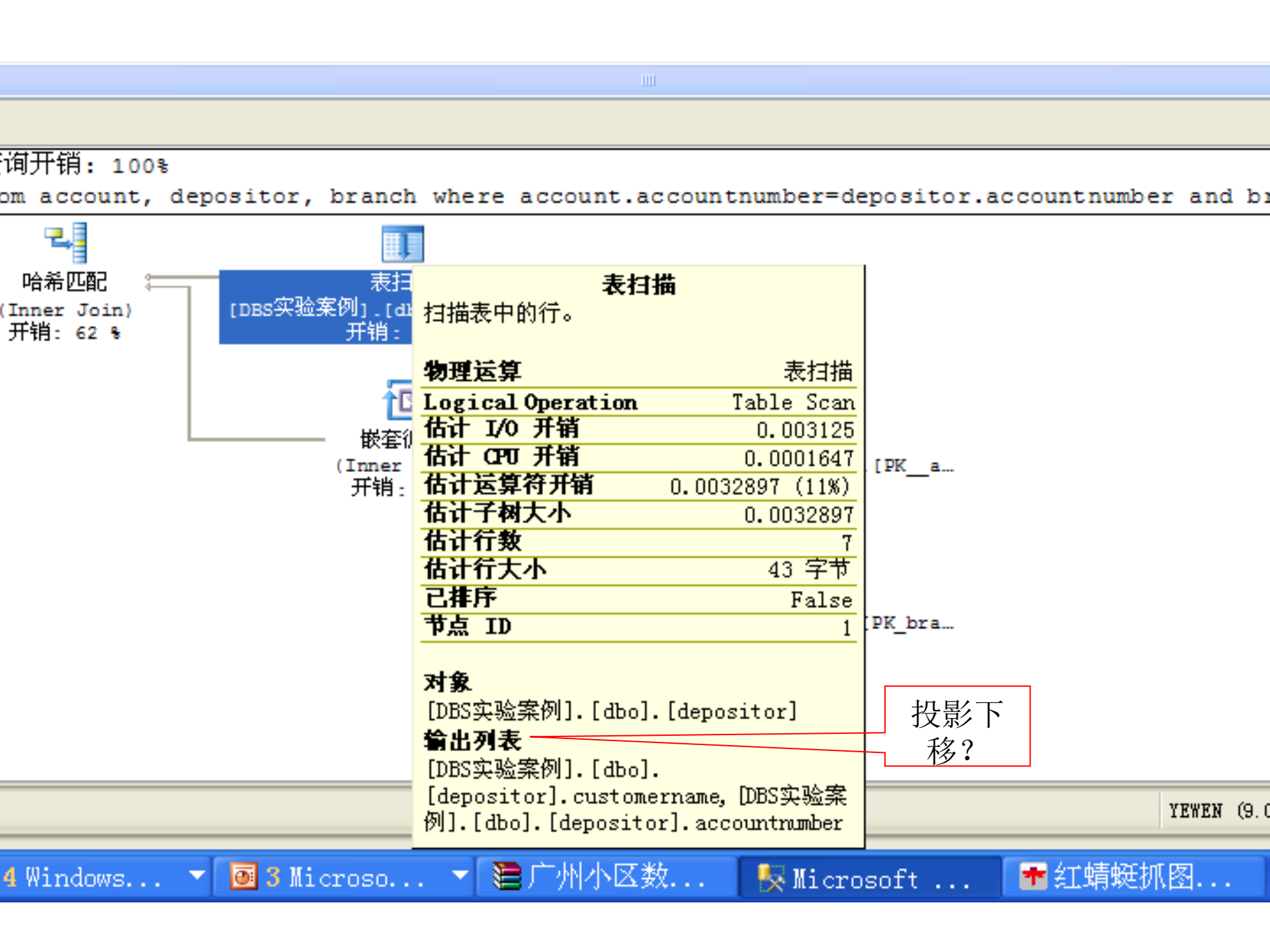


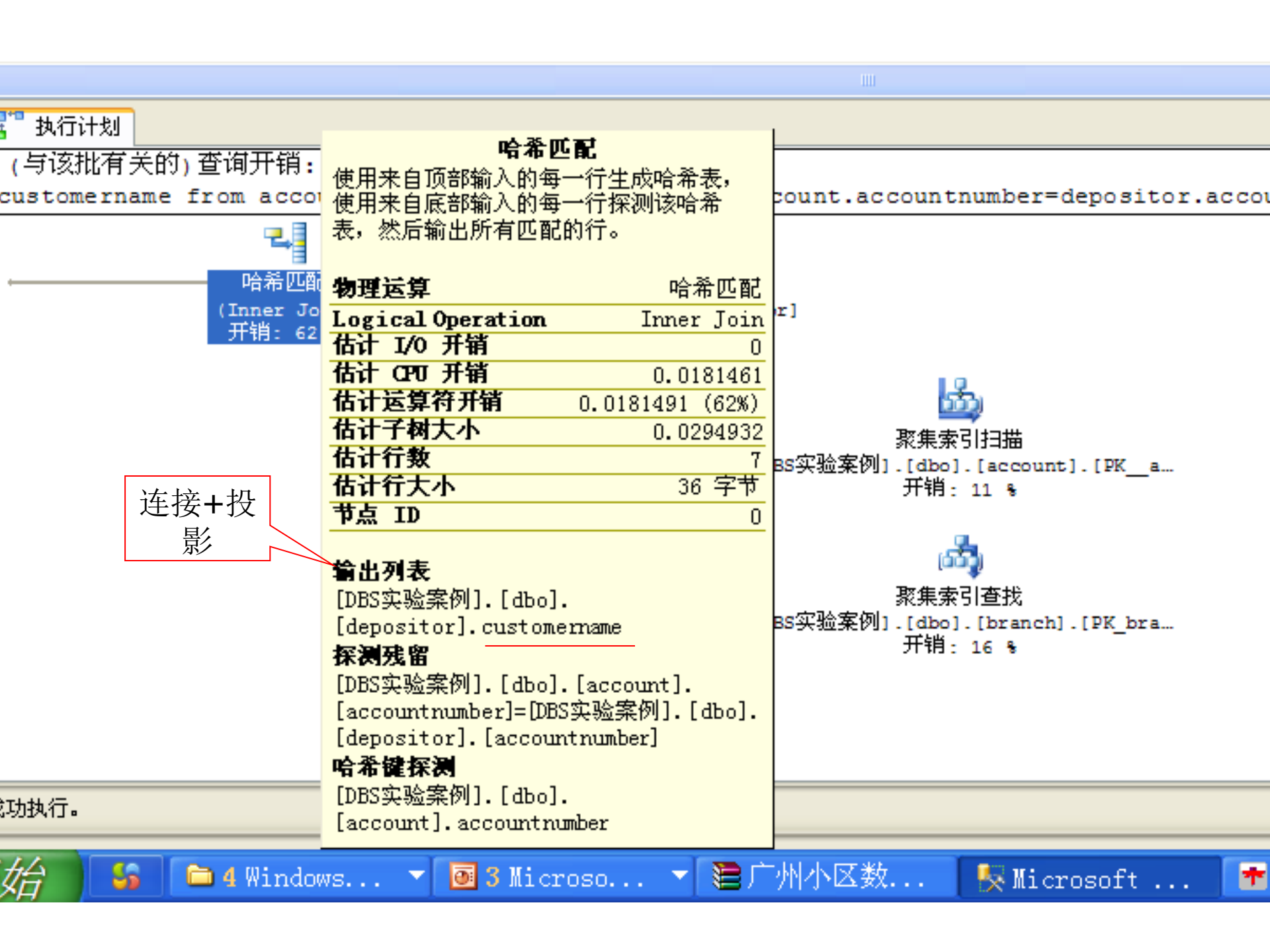
```
r=depositor.accountnumber
account.branchname
yn'
balance<1000
```




r, branch where account.accountnur
anch.branch
表扫描
验案例].[dbo].[depositor]
开销: 11 %
嵌套循环
(Inner Join)
开销: 0 %
聚集索引扫
[DBS实验案例].[dbo].[a
开销: 11
聚集索引查
[DBS实验案例].[dbo].[b
开销: 16
聚集索引查找
扫描聚集索引中特定范围的行。
物理运算 聚集索引查找
Logical Operation Clustered Index Seek
估计 I/O 开销 0.003125
估计 CPU 开销 0.0001581
估计运算符开销 0.004706 (16%)
估计子树大小 0.004706
估计行数 1
估计行大小 29 字节
已排序 True
节点 ID 4
谓词
[DBS实验案例].[dbo].[branch].[assets]>
(100000.) AND [DBS实验案例].[dbo].
[branch].[branchcity]='Brooklyn'
对象
[DBS实验案例].[dbo].[branch].[PK_branch]
输出列表
[DBS实验案例].[dbo].[branch].branchcity,
[DBS实验案例].[dbo].[branch].assets
Seek 谓词
前缀: [DBS实验案例].[dbo].
[branch].branchname = [DBS实验案例].
[dbo].[account].[branchname]
选择下移
投影下移?
RTM)
YEWEN\y
Microso...
广州小区数...
Microsoft ...
红蜻蜓抓图...
65%


询开销： 100%
m account, depositor, branch where account.accountnumber=depositor.accountnumber and
哈希匹配
Inner Join)
开销： 62 %
表
[DBS实验案例].[
开销
嵌套循环
对于顶部(外部)输入的每一行，扫描底部(内部)输入，然后输出匹配的行。
物理运算 嵌套循环
Logical Operation Inner Join
估计 I/O 开销 0
估计 CPU 开销 0.0000418
估计运算符开销 0.0000554 (0%)
估计子树大小 0.0080544
估计行数 10
估计行大小 16 字节
节点 ID 2
输出列表
[DBS实验案例].[dbo].[account].accountnumber
外部引用
[DBS实验案例].[dbo].[account].branchname
连接实现方式
投影下移
YEWEN
Windows...
3 Microso...
广州小区数...
Microsoft ...
红蜻蜓抓图...


询开销： 100%
om account, depositor, branch where account.accountnumber=depositor.accountnumber and br
哈希匹配
(Inner Join)
开销: 62 %
表扫
[DBS实验案例].[d
开销:
嵌套
(Inner
开销:
表扫描
扫描表中的行。
物理运算 表扫描
Logical Operation Table Scan
估计 I/O 开销 0.003125
估计 CPU 开销 0.0001647
估计运算符开销 0.0032897 (11%)
估计子树大小 0.0032897
估计行数 7
估计行大小 43 字节
已排序 False
节点 ID 1
对象
[DBS实验案例].[dbo].[depositor]
输出列表
[DBS实验案例].[dbo].[depositor].customername, [DBS实验案例].[dbo].[depositor].accountnumber
[PK__a...
[PK_bra...
投影下移？
YEWEN (9.0
4 Windows...
3 Microso...
广州小区数...
Microsoft ...
红蜻蜓抓图...


执行计划
(与该批有关的)查询开销:
customername from accou
count.accountnumber=depositor.accou
哈希匹配
(Inner Jo
开销: 62
哈希匹配
使用来自顶部输入的每一行生成哈希表，使用来自底部输入的每一行探测该哈希表，然后输出所有匹配的行。
物理运算 哈希匹配
Logical Operation Inner Join
估计 I/O 开销 0
估计 CPU 开销 0.0181461
估计运算符开销 0.0181491 (62%)
估计子树大小 0.0294932
估计行数 7
估计行大小 36 字节
节点 ID 0
输出列表
[DBS实验案例].[dbo].[depositor].customername
探测残留
[DBS实验案例].[dbo].[account].[accountnumber]=[DBS实验案例].[dbo].[depositor].[accountnumber]
哈希键探测
[DBS实验案例].[dbo].[account].accountnumber
连接+投影
聚集索引扫描
BS实验案例].[dbo].[account].[PK__a...
开销: 11 %
聚集索引查找
BS实验案例].[dbo].[branch].[PK_bra...
开销: 16 %
成功执行。
始
4 Windows...
3 Microso...
广州小区数...
Microsoft ...

Example Four (cont.)

给定如下关系数据库

Employee(Employee#, Name, Address, Super-E#, E-D#)

Department(D#, Dname, manager#, depart-location)

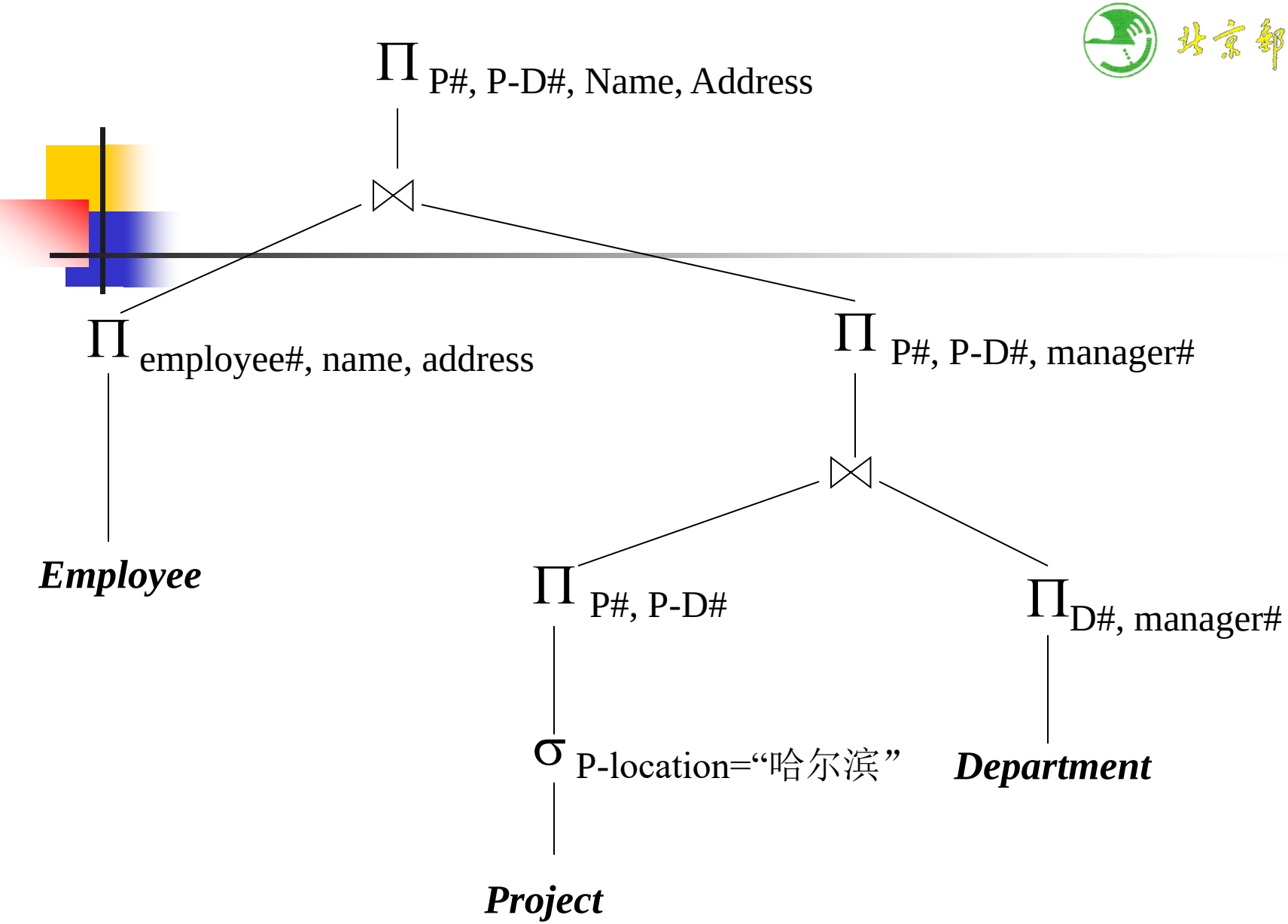
Project(P#, Pname, Plocation, P-D#)

- 查询：对于每个在“哈尔滨”进行的工程项目，列出其工程项目号P#，工程所属部门号P-D#, 该部门领导的名字Name和地址Address
- 要求：写出该查询的SQL语句；转换为关系代数表达式并利用启发式方法进行查询优化；给出优化后的关系代数表达式。
- Select P#, P-D#, Name, Address)

  From Project, Department, Employee

  Where Plocation =哈尔滨 AND P-D#= D#

  AND manager# = Employee#


Π P#, P-D#, Name, Address
⋈
Π employee#, name, address
Π P#, P-D#, manager#
Employee
⋈
Π P#, P-D#
Π D#, manager#
σ P-location="哈尔滨"
Department
Project

作业1

- Consider the following schema, where the primary keys are underlined ,
- Suppliers (supplier-id, supplier-name, city, telephone, address)
- Parts (parts-id, parts-name, parts-color)
- Catalog (supplier-id, parts-id, price)
- .
- (1) Give an SQL statement to find out the *name* and *telephone* of the suppliers who supply a red *part* whose *price* is below $2000.
- (2) Translate this SQL statement into an initial query tree, and give an optimized query *tree* for it, by means of heuristic query optimization.

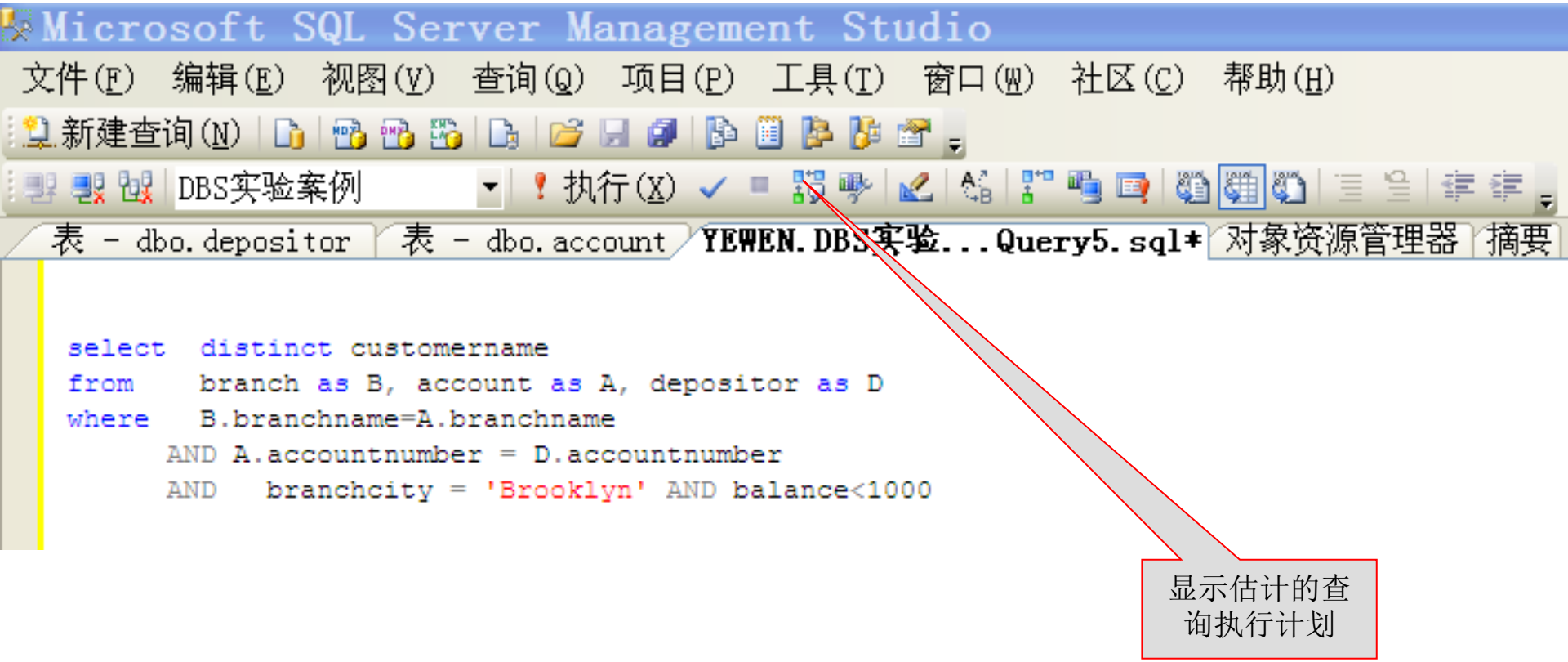
作业2

- Consider the database University given in the textbook,

- (1) Give a SQL statement to find some students and list their names and departments that they belong to. It is required that their total credits (presented by tot_cred) are more than 40, their departments are located in Building 3, and they take the course identified by ‘2016CSDBS’. (3 points)

- (2) For the SQL statement in (1), give an optimized query tree. (7 points)

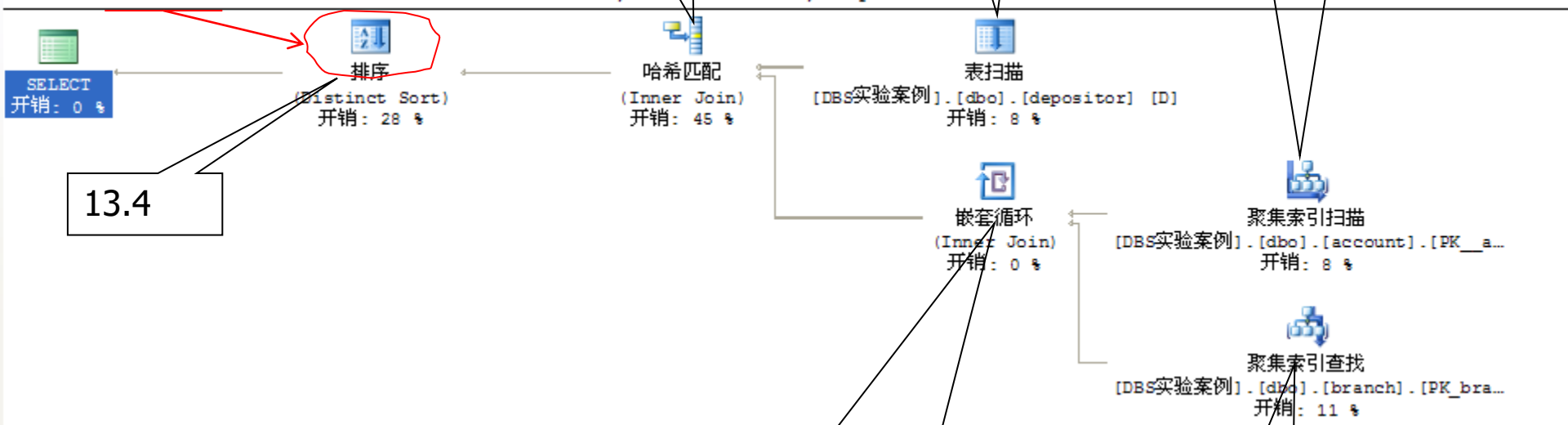
Appendix A SQL Server平台下观察比较SQL语句查询执行计划



13.5.5 hash join，
Fig. 13.8

A1算法；
outer relation

A1算法；
outer relation

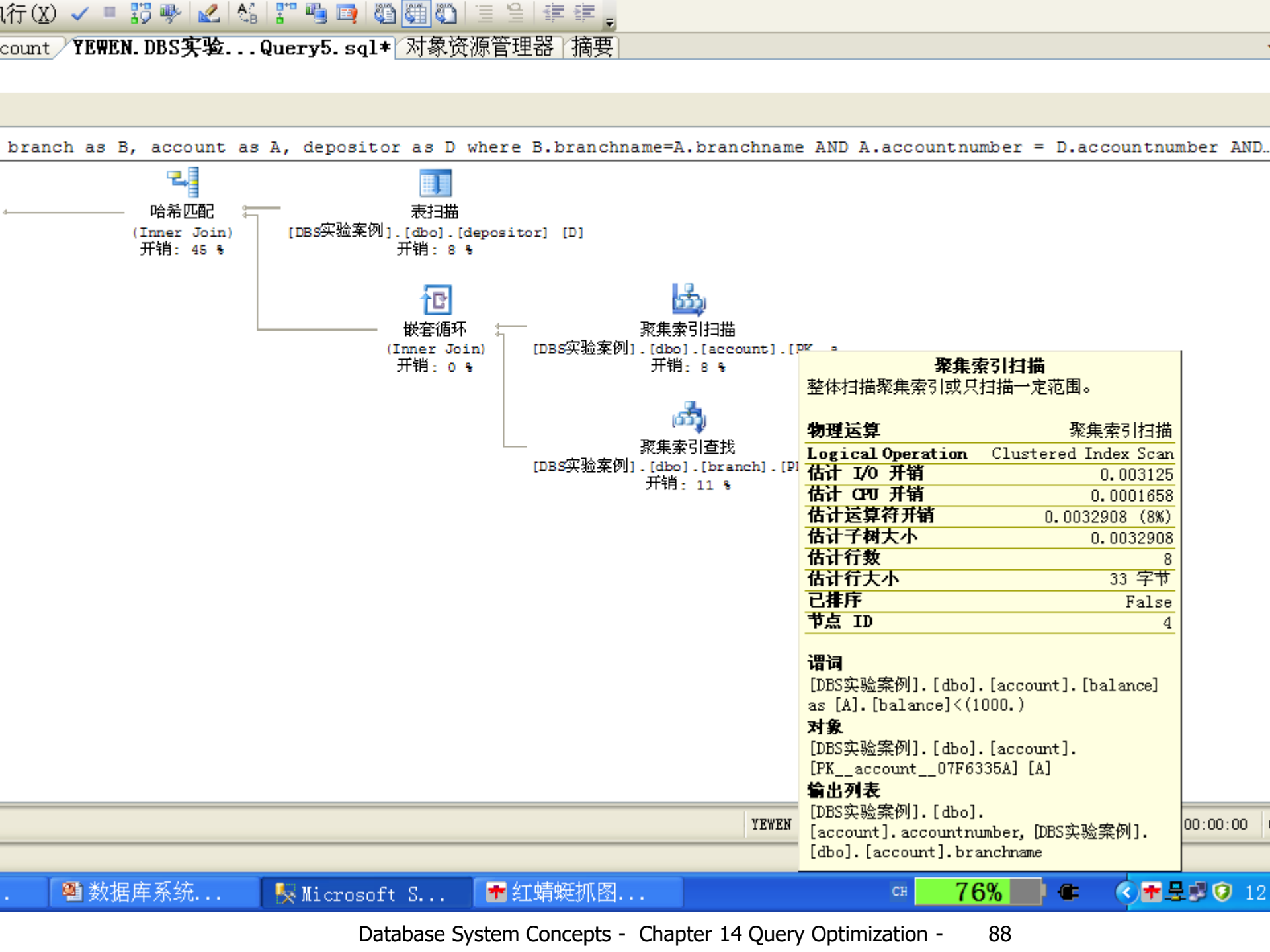


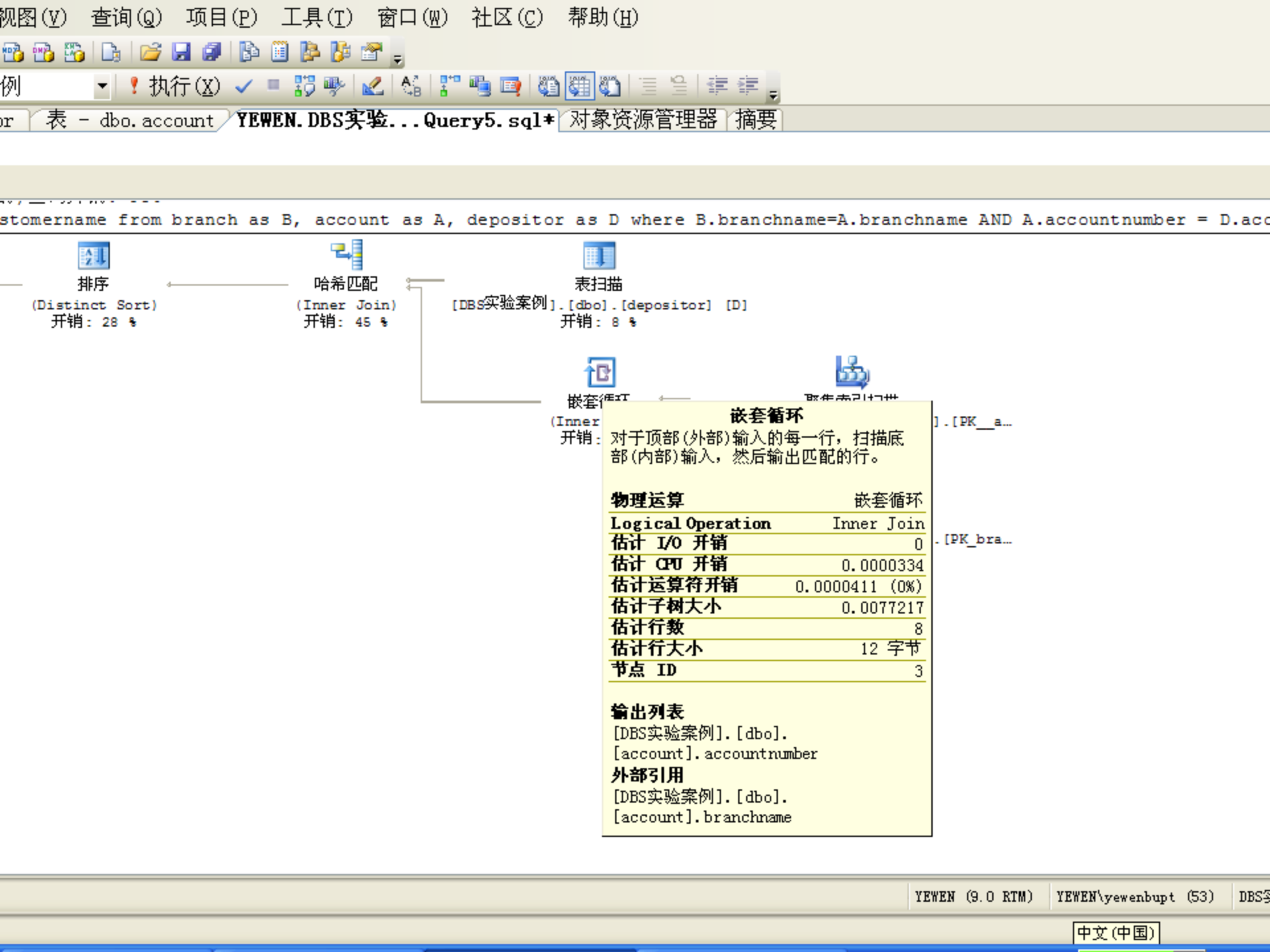
13.4

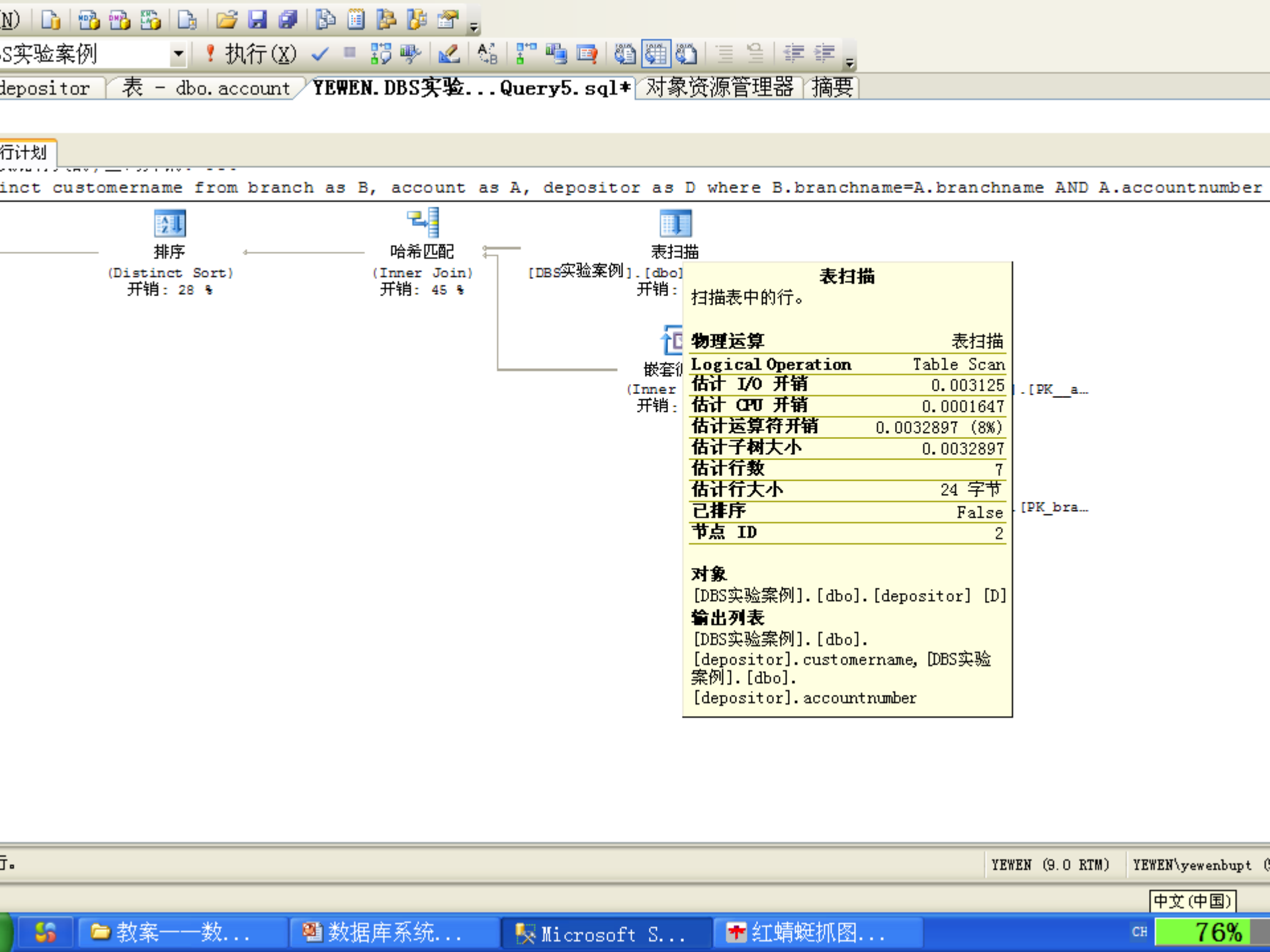
13.5.2 indexed nested loop join，
在内关系branch的连接属性branchname上建有索引

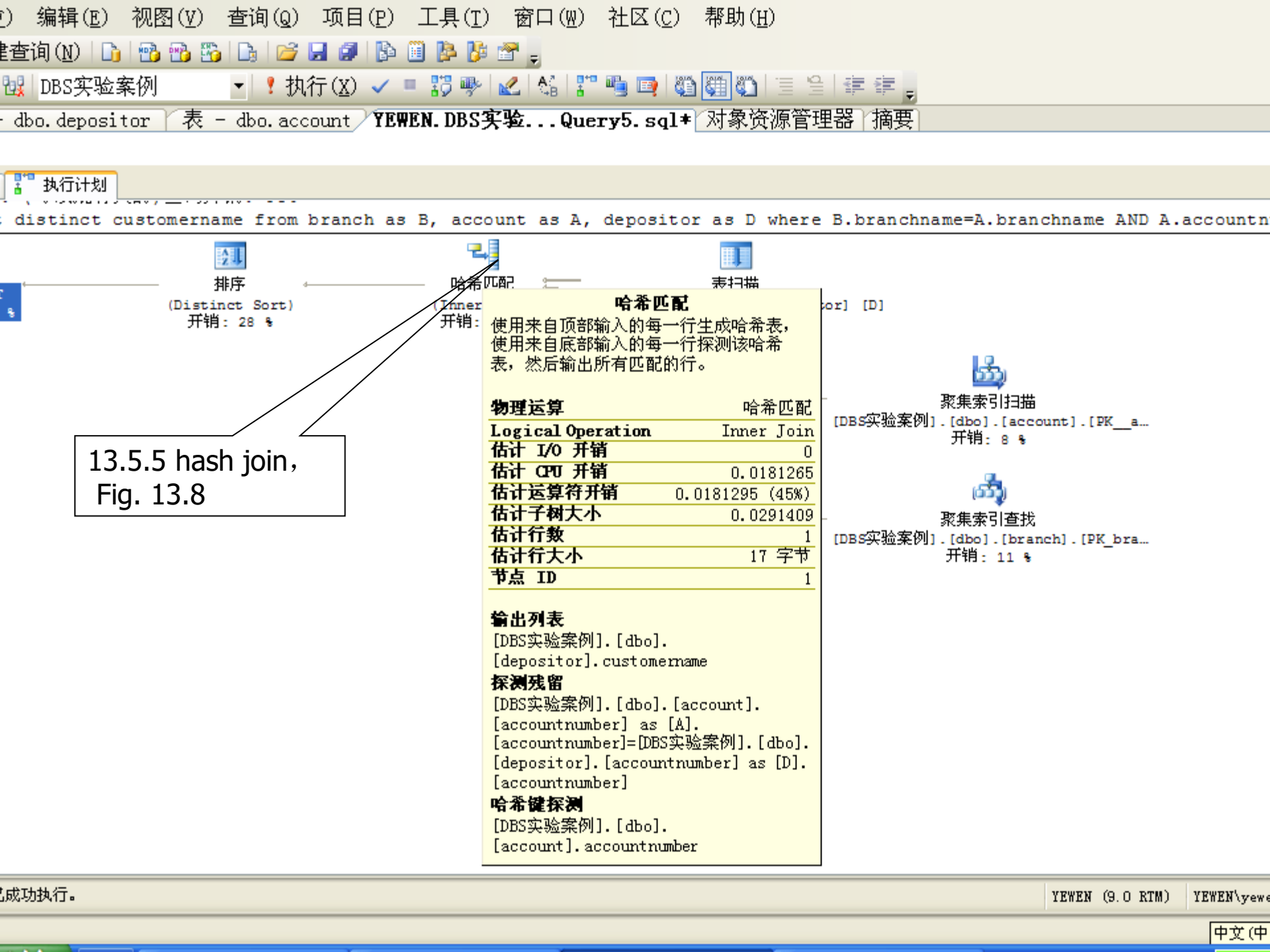
13.3.2 A3算法；
inner relation, **holded in *main memory***

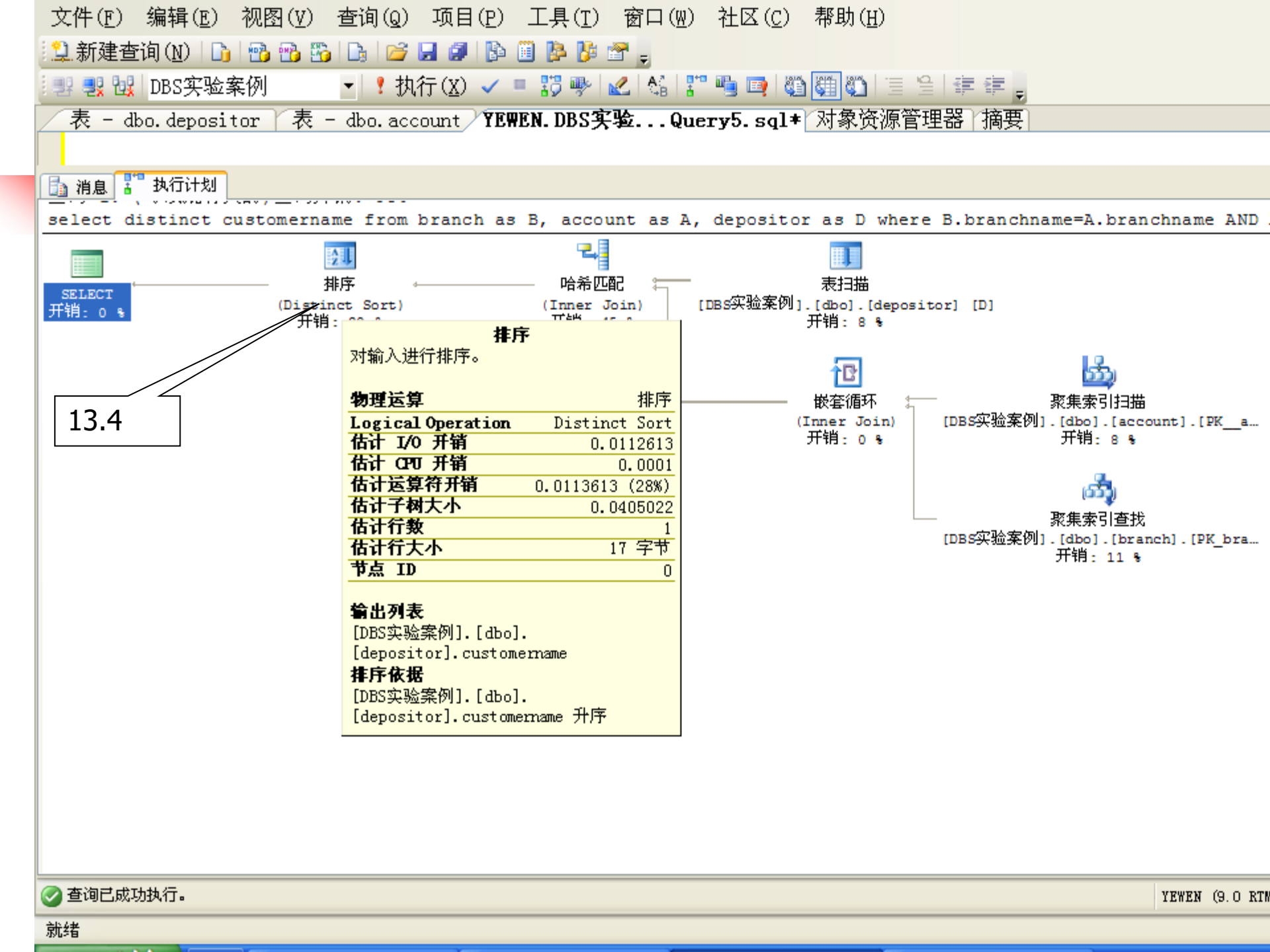

ch as B, account as A, depositor as D where B.branchname=A.branchname AND A.accountnumber = D.accountnumber AND…
哈希匹配
(Inner Join)
开销: 45 %
表扫描
[DBS实验案例].[dbo].[depositor] [D]
开销: 8 %
嵌套循环
(Inner Join)
开销: 0 %
聚集索引扫描
[DBS实验案例].[dbo].[account].[PK__a…
开销: 8 %
聚集索引查找
[DBS实验案例].[dbo].[branch].[PK_bra…
开销: 11 %
聚集索引查找
扫描聚集索引中特定范围的行。
物理运算 聚集索引查找
Logical Operation Clustered Index Seek
估计 I/O 开销 0.003125
估计 CPU 开销 0.0001581
估计运算符开销 0.0043898 (11%)
估计子树大小 0.0043898
估计行数 1
估计行大小 20 字节
已排序 True
节点 ID 5
谓词
[DBS实验案例].[dbo].[branch].[branchcity] as [B].[branchcity]='Brooklyn'
对象
[DBS实验案例].[dbo].[branch].[PK_branch] [B]
输出列表
[DBS实验案例].[dbo].[branch].branchcity
Seek 谓词
前缀: [DBS实验案例].[dbo].[branch].branchname=[DBS实验案例].[dbo].[account].[branchname] as [A].[branchname]
YEWEN (9.0
:00
0 行
行 5
Ins
数据库系统...
Microsoft S...
红蜻蜓抓图...
捕捉预览 - ...
CH
76%
12:41


count
YEWEN.DBS实验...Query5.sql*
对象资源管理器
摘要
branch as B, account as A, depositor as D where B.branchname=A.branchname AND A.accountnumber = D.accountnumber AND
哈希匹配
(Inner Join)
开销: 45 %
表扫描
[DBS实验案例].[dbo].[depositor] [D]
开销: 8 %
嵌套循环
(Inner Join)
开销: 0 %
聚集索引扫描
[DBS实验案例].[dbo].[account].[PK
开销: 8 %
聚集索引查找
[DBS实验案例].[dbo].[branch].[P
开销: 11 %
聚集索引扫描
整体扫描聚集索引或只扫描一定范围。
物理运算 聚集索引扫描
Logical Operation Clustered Index Scan
估计 I/O 开销 0.003125
估计 CPU 开销 0.0001658
估计运算符开销 0.0032908 (8%)
估计子树大小 0.0032908
估计行数 8
估计行大小 33 字节
已排序 False
节点 ID 4
谓词
[DBS实验案例].[dbo].[account].[balance] as [A].[balance]<(1000.)
对象
[DBS实验案例].[dbo].[account].[PK__account__07F6335A] [A]
输出列表
[DBS实验案例].[dbo].[account].accountnumber, [DBS实验案例].[dbo].[account].branchname
YEWEN
00:00:00
数据库系统...
Microsoft S...
红蜻蜓抓图...
CH
76%


视图(V) 查询(Q) 项目(P) 工具(T) 窗口(W) 社区(C) 帮助(H)
执行(X)
表 - dbo.account
YEWEN.DBS实验...Query5.sql*
对象资源管理器
摘要
stomername from branch as B, account as A, depositor as D where B.branchname=A.branchname AND A.accountnumber = D.acc
排序
(Distinct Sort)
开销: 28 %
哈希匹配
(Inner Join)
开销: 45 %
表扫描
[DBS实验案例].[dbo].[depositor] [D]
开销: 8 %
嵌套循环
对于顶部(外部)输入的每一行，扫描底部(内部)输入，然后输出匹配的行。
物理运算 嵌套循环
Logical Operation Inner Join
估计 I/O 开销 0
估计 CPU 开销 0.0000334
估计运算符开销 0.0000411 (0%)
估计子树大小 0.0077217
估计行数 8
估计行大小 12 字节
节点 ID 3
输出列表
[DBS实验案例].[dbo].[account].accountnumber
外部引用
[DBS实验案例].[dbo].[account].branchname
YEWEN (9.0 RTM)
YEWEN\yewenbupt (53)
中文(中国)


S实验案例
执行(X)
depositor
表 - dbo.account
YEWEN.DBS实验...Query5.sql*
对象资源管理器
摘要
行计划
inct customername from branch as B, account as A, depositor as D where B.branchname=A.branchname AND A.accountnumber
排序
(Distinct Sort)
开销: 28 %
哈希匹配
(Inner Join)
开销: 45 %
表扫描
[DBS实验案例].[dbo]
开销:
嵌套
(Inner
开销:
表扫描
扫描表中的行。
物理运算 表扫描
Logical Operation Table Scan
估计 I/O 开销 0.003125
估计 CPU 开销 0.0001647
估计运算符开销 0.0032897 (8%)
估计子树大小 0.0032897
估计行数 7
估计行大小 24 字节
已排序 False
节点 ID 2
对象
[DBS实验案例].[dbo].[depositor] [D]
输出列表
[DBS实验案例].[dbo].
[depositor].customername, [DBS实验
案例].[dbo].
[depositor].accountnumber
YEWEN (9.0 RTM)
YEWEN\yewenbupt
中文(中国)
教案——数...
数据库系统...
Microsoft S...
红蜻蜓抓图...
CH
76%


编辑(E) 视图(V) 查询(Q) 项目(P) 工具(T) 窗口(W) 社区(C) 帮助(H)
DBS实验案例
执行(X)
dbo.depositor
表 - dbo.account
YEWEN.DBS实验...Query5.sql*
对象资源管理器
摘要
执行计划
distinct customername from branch as B, account as A, depositor as D where B.branchname=A.branchname AND A.accountn
排序
(Distinct Sort)
开销: 28 %
哈希匹配
开销:
表扫描
哈希匹配
使用来自顶部输入的每一行生成哈希表，使用来自底部输入的每一行探测该哈希表，然后输出所有匹配的行。
物理运算 哈希匹配
Logical Operation Inner Join
估计 I/O 开销 0
估计 CPU 开销 0.0181265
估计运算符开销 0.0181295 (45%)
估计子树大小 0.0291409
估计行数 1
估计行大小 17 字节
节点 ID 1
输出列表
[DBS实验案例].[dbo].[depositor].customername
探测残留
[DBS实验案例].[dbo].[account].[accountnumber] as [A].[accountnumber]=[DBS实验案例].[dbo].[depositor].[accountnumber] as [D].[accountnumber]
哈希键探测
[DBS实验案例].[dbo].[account].accountnumber
聚集索引扫描
[DBS实验案例].[dbo].[account].[PK__a...
开销: 8 %
聚集索引查找
[DBS实验案例].[dbo].[branch].[PK_bra...
开销: 11 %
13.5.5 hash join，
Fig. 13.8
成功执行。
YEWEN (9.0 RTM)
YEWEN\yewe
中文(中


文件(F) 编辑(E) 视图(V) 查询(Q) 项目(P) 工具(T) 窗口(W) 社区(C) 帮助(H)
新建查询(N)
DBS实验案例
执行(X)
表 - dbo.depositor
表 - dbo.account
YEWEN.DBS实验...Query5.sql*
对象资源管理器
摘要
消息
执行计划
select distinct customername from branch as B, account as A, depositor as D where B.branchname=A.branchname AND
SELECT
开销: 0 %
排序
(Distinct Sort)
哈希匹配
(Inner Join)
表扫描
[DBS实验案例].[dbo].[depositor] [D]
开销: 8 %
13.4
排序
对输入进行排序。
物理运算 排序
Logical Operation Distinct Sort
估计 I/O 开销 0.0112613
估计 CPU 开销 0.0001
估计运算符开销 0.0113613 (28%)
估计子树大小 0.0405022
估计行数 1
估计行大小 17 字节
节点 ID 0
输出列表
[DBS实验案例].[dbo].
[depositor].customername
排序依据
[DBS实验案例].[dbo].
[depositor].customername 升序
嵌套循环
(Inner Join)
开销: 0 %
聚集索引扫描
[DBS实验案例].[dbo].[account].[PK__a...
开销: 8 %
聚集索引查找
[DBS实验案例].[dbo].[branch].[PK_bra...
开销: 11 %
查询已成功执行。
YEWEN (9.0 RTM
就绪

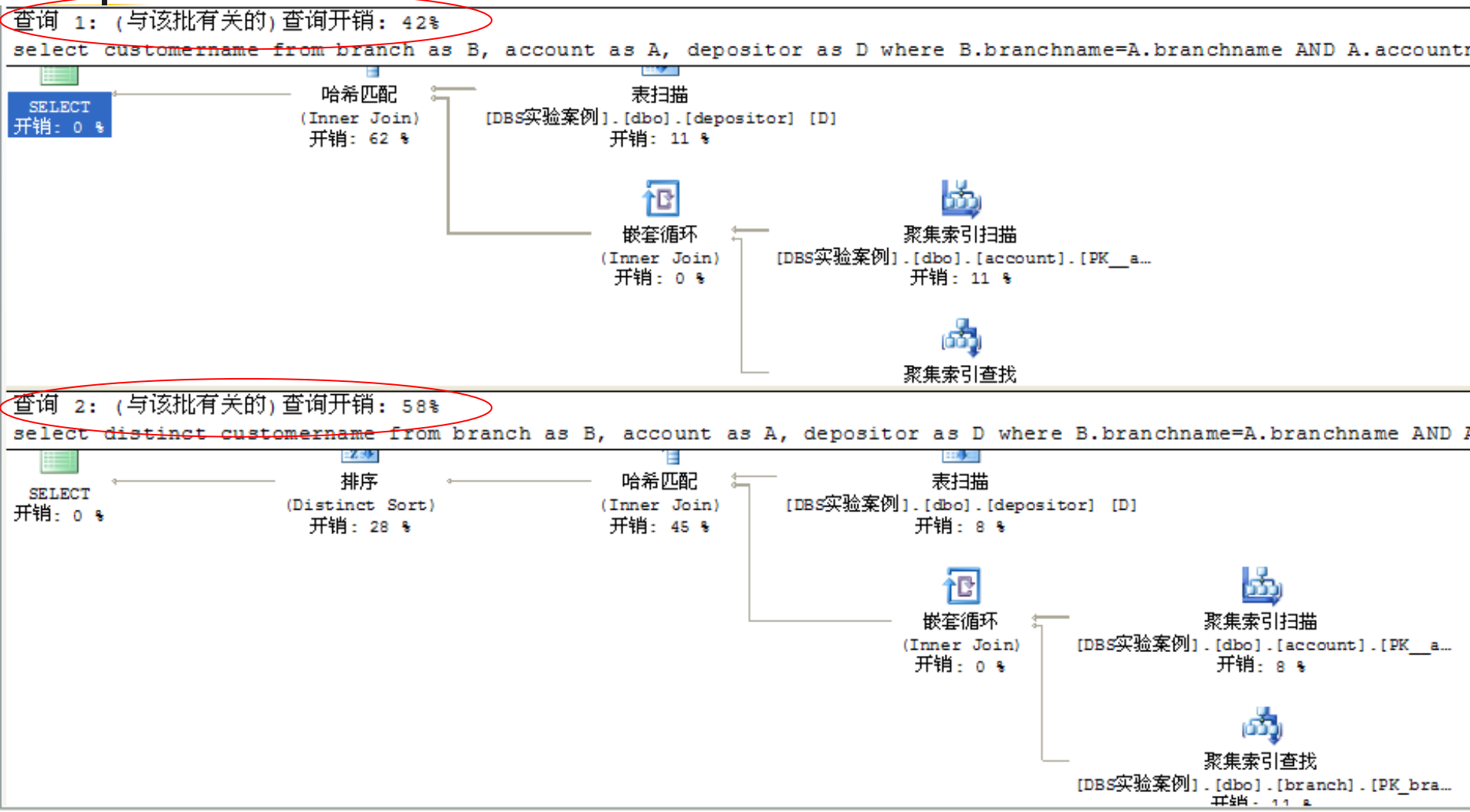
比较2条SQL语句查询成本

同时提交1批（2条）查询语句：

```
select  distinct customername
from    branch as B, account as A, depositor as D
where   B.branchname=A.branchname
      AND A.accountnumber = D.accountnumber
      AND   branchcity = 'Brooklyn' AND balance<1000

select  customername
from    branch as B, account as A, depositor as D
where   A.accountnumber = D.accountnumber
      AND B.branchname=A.branchname
      AND   branchcity = 'Brooklyn' AND balance<1000
```

该批次中2条查询语句的成本之比： 42% vs. 58%



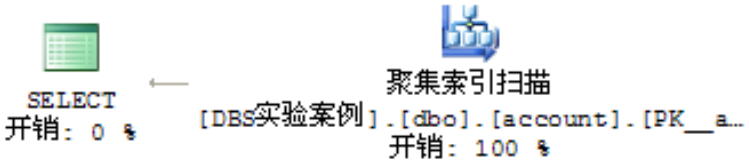
```
select *
from account
where  balance >= 0 and  balance <= 50


select *
from account
where balance in (
            select  balance
            from    account
            where  balance >= 0 and  balance <= 50)
```

消息 执行计划

查询 1: (与该批有关的)查询开销: 33%

```
select * from account where balance >= 0 and balance <= 50
```



查询 2: (与该批有关的)查询开销: 67%

```
select * from account where balance in ( select balance from account where balance >= 0 and balance <
```